REFERENCE

El-Barookah El-Moharrama

Defines 4
- C++ 4
- JAVA 4
- Python 4

Algebra Formulas 5
Geometry Formulas 6
Combinatorics Formulas 8
Algorithms 13
Geometry 13
- Dot product 13
- Cross product 13
- dcmp 13
- Fix Angle 14
- Cos Rule 14
- Perpendicular Points 14
- Is collinear 14
- Point rotation 14
- AngleAOB 15
- Orient (CCW test) 15
- Polygon area 15
- Is Polygon convex 15
- Reflection point 16
- Segments intersection 17
- Equal lines (segments) 17
- Sort_CCW 17
- Convex Hull 18
- Convex Hull Complex 20
- Parametric line equation 21
- Is Point On Ray 22

Point to Line distance 22
Point to Segment distance 22
Line Lattice Points Count 22
Line Line Intersection 23
Circumcircle 23
Circle Line Intersections 24
CircleCircle Intersection 25
Minimum enclosing circle MEC (untested) 25
Polygon centroid 26
Point in polygon 27
Point in CONVEX polygon multiple queries 27
Pick’s Theorem (Count number of internal integer points) 31
Area of union rectangles 31
Algebra 35
Miller Rabin 35
GCD 36
Extended GCD 36
Fast Power 36
Sieve 37
Fast Fibonacci 37
Mod Inverse (using eGCD) 37
Mod Inverse (using Fast Power) 39
Pre Inverse 39
Matrix Multiplication 39
Permutations 40
Matrix inverse 40
Find Any Solution 41
Shifting Solution 41
Find All Solutions 41
CRT 43
Phi Sieve 44
Phi 44
Mobius Sieve 45
Mobius 45
Burnside Lemma 46
Catalan Numbers 46
Pascals Triangle 48
Prime numbers 49
Graphs 49
K-th Shortest path 49
Floyd 50
Tarjan 52
Prim 53
Centroid 54

Check Odd Cycle 55
Euler Path 55
Centroid Decomposition 56
Finding a negative cycle in the graph 58
DSU on trees 59
LCA 60
LCA (O(N) processing, O(1) query) 61
Second Best Mst 63
Bellman-ford 68
Articulation points 69
Bridges 70
Mo on trees 72
Euler shit 74
Strings 75
Trie 75
Trie (Better Time Complexity, Worst Memory) 76
Manacher (number of palindromes in a string) 77
KMP 79
Z-Function 79
Aho-corasick 80
Suffix Automaton 83
KMP Palindrome 88
Hashing 89
Suffix Array 92
Z-Algorithm 94
KMP Pre (automaton) 95
DFA 95
Count Unique Substrings 97
DP 98
LIS 98
Matrix Power 98
CHT 99
Data Structures 101
Segment Tree 101
Segment Tree Lazy 102
Persistent Segment Tree 104
Monotonic Queue 105
Fenwick Tree 106
Sparse Table 107
2D Sparse Table 108
DSU 109
DSU Bipartite 109
DSU Rollback 111
MO 114

MEX 116
Note 116
Articles 118
Aho corasick applications 118
Suffix automaton applications 119
Number of different substrings 124
MISC. 125
Hungarian Algorithm 125
Max Flow / Min Cut 128
Hopcroft-Karp Algorithm 130
Kuhn Algorithm 132
Min Cost Max Flow 133
Probability 135
Ncr Iterative (Without Mod) 135
Notes and rules 135
Conditional Probability 136
Expected Value 136
Linear Expextation 136
Infinite Probability 136

Defines

C++

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef long long ll;
#define sfl(x) scanf("%I64d", &x)
#define sff(x) scanf("%f", &x)
#define sfd(x) scanf("%lf", &x)
#define sfc(x) scanf("%c", &x)
#define sfs(s) scanf("%size_s", &s);
#define pi (2 * acos(0))
#define wtf(s) freopen((s), "w", stdout)
#define rff(s) freopen((s), "r", stdin)
#define VIC cout.tie(0);cin.tie(0);ios_base::sync_with_stdio(0);
template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
template<typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag,
tree_order_statistics_node_update>;
Mt19937
rng(std::chrono::system_clock::now().time_since_epoch().count());
```

JAVA

```
public class solve
{
    public static void main(String args[]) throws IOException
    {
        Scanner in = new Scanner(System.in);
    }
}
FileReader FR = new FileReader("oddeven.in");
BufferedReader BR = new BufferedReader(FR);
PrintWriter pw = new PrintWriter("oddeven.out");
String s = BR.readLine();
pw.flush(); pw.close();
```

Python

```
a, b = input().split()
```

```
r, d = map(int, input().split())
a = [[int(x) for x in input().split()] for y in range(n)]
import sys
sys.stdin = open('input.txt', 'r')
sys.stdout = open('output.txt', 'w')
```

Algebra Formulas

If a and r are real numbers and $r \neq 0$, then

$$\sum_{j=0}^{n} ar^j = \begin{cases} \dfrac{ar^{n+1} - a}{r - 1} & \text{if } r \neq 1 \\ (n+1)a & \text{if } r = 1. \end{cases}$$

$\sum_{k=1}^{n} k$	$\frac{n(n+1)}{2}$
$\sum_{k=1}^{n} k^2$	$\frac{n(n+1)(2n+1)}{6}$
$\sum_{k=1}^{n} k^3$	$\frac{n^2(n+1)^2}{4}$
$\sum_{k=0}^{\infty} x^k, \lvert x\rvert < 1$	$\frac{1}{1-x}$
$\sum_{k=1}^{\infty} kx^{k-1}, \lvert x\rvert < 1$	$\frac{1}{(1-x)^2}$

- **LCM * GCD** = the multiple of the 2 numbers
- **LCM** = multiply the numbers which has the greatest power from the 2 numbers even if they're not common
- **GCD** = multiply the numbers which has the lowest power from the 2 numbers even if they're not common

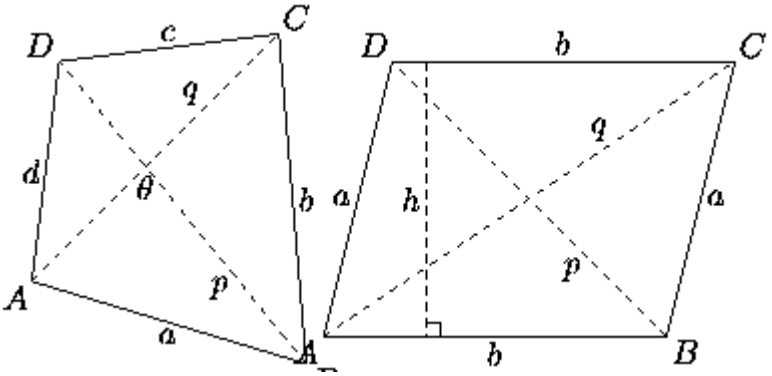
- **Arithmetic sequence :** $\frac{n}{2}(a + b)$
- **Geometric sequence :** $\frac{L * R - a}{r - 1} = a * \frac{r^n - 1}{r - 1}$
- **Sum of odd numbers :** $\left(\frac{n + n \% 2}{2}\right)^2$

- **Sum of even numbers:** $\dfrac{(\frac{n}{2} * (2 + n - n \% 2))}{2}$

- **Quadratic Formula :** $\dfrac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

Geometry Formulas

- **Number of diagonals =** $\dfrac{n(n-3)}{2}$



- **Area =** $\frac{1}{2} PQ\, sin(\theta)$**=**$\frac{1}{4}(b^2 + d^2 - a^2 + c^2) tan(\theta)$**=**

  $\frac{1}{4}\sqrt{4P^2Q^2 - (b^2 + d^2 - a^2 - c^2)^2}$ **=**

  $\sqrt{(s - a)(s - b)(s - c)(s - d) - abcdcos\frac{1}{2}(A + C)}$

- **Area of sector =** $\frac{1}{2} r^2 \theta_{rad}$

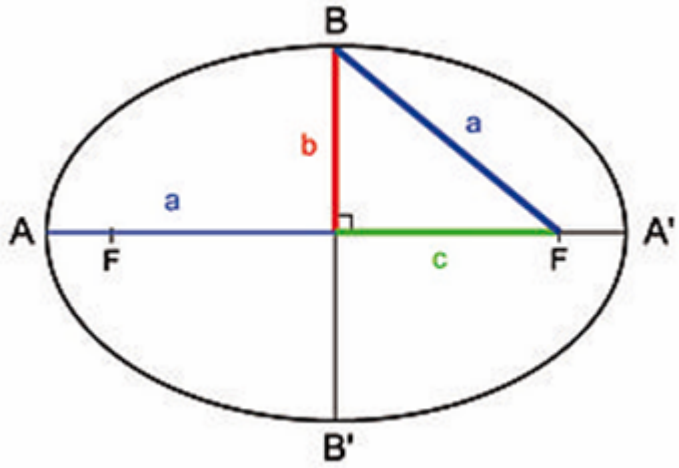
- **Area of triangle =** $\sqrt{p(p - a)(p - b)(p - c)}$**, where a, b, c are the lengths of the three sides and p =** $\frac{a+b+c}{2}$ **=**

  $\frac{absin(C)}{2}$ **=** $\left| \frac{Ax(By - Cy) + Bx(Cy - Ay) + Cx(Ay - By)}{2} \right|$ **=** $\sqrt{\frac{3}{4}} S^2$**,**

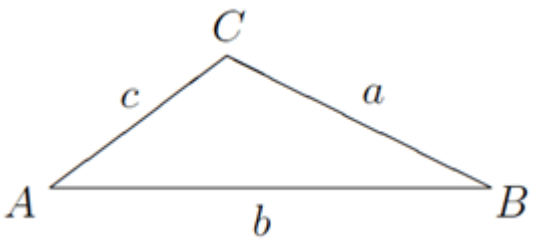
  where s is any side of an equilateral triangle =

  $\dfrac{A \times B + B \times C + C \times A}{2}$ **, where A, B, C are the three vectors**

- Cross product equals area of parallelogram

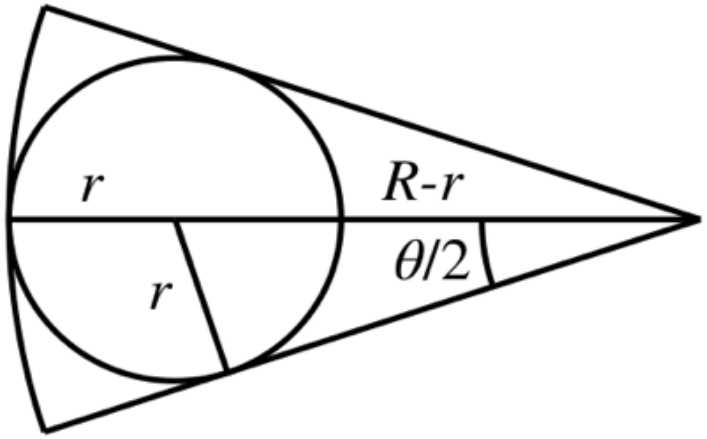


- **Area of an ellipse:** **abπ**
- **Area of trapezium:** $\frac{a+b}{2}h$
- **The volume of cone:** $\frac{h}{3}\Pi r^2$
- **The volume of Pyramid:** $\frac{abh}{3}$
- **Lengths of a right-angled triangle given non-hypot side:** $\frac{n^2}{4}, \frac{n^2}{4}+1,\ n$**,n is even,** $\frac{n^2}{2}, \frac{n^2}{2}+1,\ n$**,n is odd**
- **Chord length:** $2r\sin(\frac{\theta}{2})$



- **$c^2 = a^2+b^2-2ab\cos(C)$**

- **θ°=** $\dfrac{\theta^{rad} * 180}{\Pi}$



says

$$r = (R - r)\sin\left(\frac{\theta}{2}\right)$$

Therefore, we can solve for r:

$$\boxed{r = R\,\frac{\sin\left(\frac{\theta}{2}\right)}{1 + \sin\left(\frac{\theta}{2}\right)}}$$

To find θ from R and r, we can use

$$\sin\left(\frac{\theta}{2}\right) = \frac{r}{R - r}$$

-

Combinatorics Formulas

- $(1 + t)^n = \sum_{k=0}^{n} \binom{n}{k} t^k,\ t == 1 => 2^n$
- $(x + y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k = \sum_{k=0}^{n} x^k y^{n-k}$
- $\left(1 + \frac{1}{n}\right)^n = 1 + \binom{n}{1}\frac{1}{n} + \binom{n}{2}\frac{1}{n^2} + \binom{n}{3}\frac{1}{n^3} + \cdots + \binom{n}{n}\frac{1}{n^n}.$

- $\binom{n}{k}\frac{1}{n^k} = \frac{1}{k!} \cdot \frac{n(n-1)(n-2)\cdots(n-k+1)}{n^k}$

- Permutation in circle = $nPr\ /\ r$

- Number of ways to distribute n distinct objects around r identical boxes such that each box has at least one object are denoted by $s(n,\ r)$:

- $s(n, r) = r\ *\ s(n - 1, r) + s(n - 1, r - 1)$

1. $r > n \implies S(n, r) = 0.$
2. $r = n \implies S(n, n) = 1.$
3. $r = n - 1 \implies S(n, n - 1) = \binom{n}{2}.$
4. $r = n - 2 \implies S(n, n - 2) = \binom{n}{3} + 3\binom{n}{4}.$
5. $r = 1 \implies S(n, 1) = 1.$
6. $r = 2 \implies S(n, 2) = 2^{n-1} - 1.$

- The number of ways to reach cell $(n,\ m)$ from $(1,\ 1)$ = $(n\ +\ m)Cn$

- The number of r-combinations of $x\ =\ \{1,\ 2,\ 3,\ \ldots,\ n\}$ with no consecutive integers = $(n\ -\ r\ +\ 1)Cr$

- The number of ways to distribute n objects into m boxes, such that box i has n_i objects:

$$\binom{n}{n_1, n_2, \ldots, n_m} = \frac{n!}{n_1! n_2! \cdots n_m!}.$$

$$\binom{n}{r} = \binom{n}{n-r} \tag{2.1.1}$$

$$\binom{n}{r} = \frac{n}{r}\binom{n-1}{r-1} \quad \text{provided that } r \geq 1 \tag{2.1.2}$$

$$\binom{n}{r} = \frac{n-r+1}{r}\binom{n}{r-1} \quad \text{provided that } r \geq 1 \tag{2.1.3}$$

$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r} \tag{2.1.4}$$

$$\binom{n}{m}\binom{m}{r} = \binom{n}{r}\binom{n-r}{m-r} \tag{2.1.5}$$

$$\binom{n}{n_1, n_2, ..., n_m} = \frac{n!}{n_1! n_2! \cdots n_m!}.$$

Let p be a prime. Then

(i) $\binom{n}{p} \equiv \lfloor \frac{n}{p} \rfloor \pmod{p}$ for every $n \in \mathbf{N}$,

(ii) $\binom{p}{r} \equiv 0 \pmod{p}$ for every r such that $1 \leq r \leq p-1$,

(iii) $\binom{p+1}{r} \equiv 0 \pmod{p}$ for every r such that $2 \leq r \leq p-1$,

(iv) $\binom{p-1}{r} \equiv (-1)^r \pmod{p}$ for every r such that $0 \leq r \leq p-1$,

(v) $\binom{p-2}{r} \equiv (-1)^r (r+1) \pmod{p}$ for every r such that $0 \leq r \leq p-2$,

(vi) $\binom{p-3}{r} \equiv (-1)^r \binom{r+2}{2} \pmod{p}$ for every r such that $0 \leq r \leq p-3$.

communication) that an integer $n > 2$ is prime iff

$$n \left| \binom{n}{6k \pm 1} \right.,$$

for all k with $1 \leq k \leq \lfloor \frac{\sqrt{n}}{6} \rfloor$, where $\lfloor x \rfloor$ denotes the greatest integer not exceeding the real number x.

$$n \text{ is a prime} \quad \text{iff} \quad n \left| \binom{n}{r} \right. \text{ for all } r = 1, 2, ..., n-1.$$

Reflection Principle (RP). The number of shortest routes from P to Q that meet the line L is equal to the number of shortest routes from P' to Q.

(i) $$\binom{r}{r} + \binom{r+1}{r} + \cdots + \binom{n}{r} = \binom{n+1}{r+1} \tag{2.5.1}$$

for all $r, n \in \mathbf{N}$ with $n \geq r$;

(ii) $$\binom{r}{0} + \binom{r+1}{1} + \cdots + \binom{r+k}{k} = \binom{r+k+1}{k} \tag{2.5.2}$$

for all $r, k \in \mathbf{N}$.

$$\begin{aligned}\sum_{i=0}^{r} \binom{m}{i}\binom{n}{r-i} &= \binom{m}{0}\binom{n}{r} + \binom{m}{1}\binom{n}{r-1} + \cdots + \binom{m}{r}\binom{n}{0} \\ &= \binom{m+n}{r}. \end{aligned} \tag{2.3.5}$$

$$\sum_{r=1}^{n} r^2 \binom{n}{r} = n(n+1)2^{n-2},$$

$$\sum_{r=1}^{n} r^3 \binom{n}{r} = n^2(n+3)2^{n-3}$$

$$\sum_{r=1}^{n} r\binom{n}{r} = \binom{n}{1} + 2\binom{n}{2} + 3\binom{n}{3} + \cdots + n\binom{n}{n} = n \cdot 2^{n-1}. \tag{2.3.4}$$

(i) $\sum_{r=0}^{n}(-1)^r\binom{n}{r} = \binom{n}{0} - \binom{n}{1} + \binom{n}{2} - \cdots + (-1)^n\binom{n}{n} = 0,$ (2.3.2)

(ii) $\binom{n}{0}+\binom{n}{2}+\cdots+\binom{n}{2k}+\cdots = \binom{n}{1}+\binom{n}{3}+\cdots+\binom{n}{2k+1}+\cdots = 2^{n-1}.$(2.3.3)

- The number of derangements, where r is the number of elements selected and k is the number of fixed points:

$$D(n,r,k) \doteq \frac{\binom{r}{k}}{(n-r)!}\sum_{i=0}^{r-k}(-1)^i\binom{r-k}{i}(n-k-i)!\,.$$

Algorithms

Geometry

Dot product

```
ld dot(pt a, pt b)
{
   // return a.X * b.X + a.Y * b.Y
   return ((conj(a) * (b)).real());
}
```

Cross product

```
ld cross(pt a, pt b)
{
   // return a.X * b.Y - b.X * a.Y
   return ((conj(a) * (b)).imag());
}
```

dcmp

```
int dcmp(ld a, ld b)
{
   // a == b : 0
   // a != b : != 0
   // a > b : 1
   // a < b : -1
   // a >= b : != -1
   // a <= b : != 1
   return fabs(a - b) <= eps ? 0 : a < b ? -1 : 1;
}
```

Fix Angle

```
ld fixAngle(ld a)
{
   return (a > 1 ? 1 : ((a < -1) ? -1 : a));
}
```

Cos Rule

```
ld cosRule(ld a, ld b, ld c)
{
   ld res = (b * b + c * c - a * a) / (2 * b * c);
   if(fabs(res - 1) < eps)
       res = 1;
   if(fabs(res + 1) < eps)
       res = -1;
   return acos(res);
}
```

Perpendicular Points

```
pt perp1(pt a)
{
   return {-a.y, a.x};
}
pt perp2(pt a)
{
   return {a.y, -a.x};
}
```

Is collinear

```
bool isCollinear(pt a, pt b, pt c)
{
   return dcmp(cross(b - a, c - a), 0) == 0;
}
```

Point rotation

```
point rotateAbout(const point &p, const point &about, double radians)
{
    return (p - about) * exp(point(0, radians)) + about;
}
```

AngleAOB

```
ld angleO(pt a, pt O, pt b) /// angle(aOb)
{
   pt v1 = (a - O), v2 = (b - O);
   return acos(fixAngle(dot(v1, v2) / (abs(v1) * abs(v2))));
}
```

Orient (CCW test)

```
ld orient(pt a, pt b, pt c) // CCW test
{
   // 1 if positive (C is to the left of AB vector) CCW
   // 0 if Zero (C is on the same direction as AB vector) Collinear
   // -1 if negative (C is to the right of AB vector) CW

   return cross(b - a, c - a);
}
```

Polygon area

```
ld polygonArea(vector<pt> &v)
{
   // can be implemented using pairs though to save time
   // cross product = (x1 * y2) - (x2 * y1)

   ld area = 0;
   int lst = v.size() - 1;
   for(int i = 0; i < v.size(); i++)
       area += cross(v[lst], v[i]), lst = i;

   return fabs(area / 2.0);
}
```

Is Polygon convex

```
bool isConvex(vector<pt> &p)
{
   int n = p.size();

   p.push_back(p[0]);
   p.push_back(p[1]);

   ld cp;
```

```
    bool pos = 0, neg = 0;
    for(int i = 0; i < n; i++)
    {
        cp = cross((p[i + 1] - p[i]), (p[i + 2] - p[i]));
        pos |= (cp > eps), neg |= (cp < -eps);
    }

    p.pop_back();
    p.pop_back();

    return pos ^ neg;  /// duplicate points will fail this function

}
```

Reflection point

```
point reflect(const point &p, const point &about1, const point &about2)
{
    point z = p - about1;
    point w = about2 - about1;
    return conj(z / w) * w + about1;
}
```

Point in Angle

```
bool inAngle(pt a, pt b, pt c, pt p) // point b in angle aBc
{
    if(orient(a, b, c) < 0)
        swap(b, c);
    return orient(a, b, p) >= 0 && orient(a, c, p) <= 0;
}
```

Point on segment

```
bool pointOnSegment(pt a, pt b, pt c)
{
    // A ..... C ...... B ??
    return dcmp(abs(b - a), (abs(c - a) + abs(b - c))) == 0;
}
```

Segments intersection

```
bool segmentIntersection(pt a, pt b, pt c, pt d, pt &out)
{
   // 1 point that does not belong to {a, b, c, d}
   // Handle using pointOnSegment function
   ld oa = orient(c, d, a),
           ob = orient(c, d, b),
           oc = orient(a, b, c),
           od = orient(a, b, d);

   // Proper intersection exists if opposite signs
   if(oa * ob < 0 && oc * od < 0)
   {
       out = (a * ob - b * oa) / (ob - oa);
       return true;
   }
   return false;
}
```

Equal lines (segments)

```
bool checkEqualLines(pair<pair<int, int>, pair<int, int>> x, pair<pair<int,
int>, pair<int, int>> y)
{
    if (x == y)
        return true;
//       x1    y1           x2    y2           x3    y3          x4    y4
//p1 = (x.x1, x.y1), p2 = (x.x2, x.y2), p3 = (y.x1, y.y1), p4 =(y.x2, y.y2)
    return (x.x1 * (x.y2 - y.y1) + x.x2 * (y.y1 - x.y1) + y.x1 * (x.y1 -
x.y2) == 0
         && x.x1 * (x.y2 - y.y2) + x.x2 * (y.y2 - x.y1) + y.x2 * (x.y1 -
x.y2) == 0);
}
```

Sort_CCW

```
complex<int> cntr;
ll cp(complex<int> a, complex<int> b)
{
    return 1LL*a.real() * b.imag() - 1LL*a.imag() * b.real();
}
bool sort_ccw(complex<int> a, complex<int> b)
{
```

```
    a-=cntr;
    b-=cntr;
    if(cp(a,b) == 0)
    {
        if(a.real() != b.real())
            return a.real() < b.real();
        return a.imag() > b.imag();
    }
    return cp(a, b) > 0;
}
```

Convex Hull

```
struct pt
{
   int x, y;
   pt(int x, int y) : x(x), y(y)
   {}
   bool operator==(const pt &other) const
   {
       return x == other.x && y == other.y;
   }
};

bool cmp(pt a, pt b)
{
   return a.x < b.x || (a.x == b.x && a.y < b.y);
}

bool cw(pt a, pt b, pt c)
{
   return a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y) <
0;
}

bool ccw(pt a, pt b, pt c)
{
   return a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y) >
0;
}

void convex_hull(vector<pt> &a)
{
   if(a.size() == 1)
```

```
        return;
    sort(a.begin(), a.end(), &cmp);
    vector<pt> newA = {a[0]};
    for(int i = 1; i < a.size(); i++)
    {
        if(!(newA.back() == a[i]))
            newA.push_back(a[i]);
    }
    a = newA;
    pt p1 = a[0], p2 = a.back();
    vector<pt> up, down;
    up.push_back(p1);
    down.push_back(p1);
    for(int i = 1; i < (int) a.size(); i++)
    {
        if(i == a.size() - 1 || cw(p1, a[i], p2))
        {
            while(up.size() >= 2 && !cw(up[up.size() - 2],
up[up.size() - 1], a[i]))
                up.pop_back();
            up.push_back(a[i]);
        }
        if(i == a.size() - 1 || ccw(p1, a[i], p2))
        {
            while(down.size() >= 2 && !ccw(down[down.size() - 2],
down[down.size() - 1], a[i]))
                down.pop_back();
            down.push_back(a[i]);
        }
    }
    a.clear();
    for(int i = 0; i < (int) down.size(); i++)
        a.push_back(down[i]);
    for(int i = up.size() - 2; i > 0; i--)
        a.push_back(up[i]);
}
```

Convex Hull Complex

```
pt pivot;
bool sortCCW(const pt &a, const pt &b)
{
   if(dcmp(cross(a - pivot, b - pivot), 0) == 0)
   {
       if(dcmp(a.imag(), b.imag()) != 0)
       {
           return a.imag() < b.imag();
       }
       return a.real() < b.real();
   }
   return cross(a - pivot, b - pivot) >= eps;
}

bool cmp(const pt &a, const pt &b)
{
   if(dcmp(a.real(), b.real()) != 0)
       return a.real() < b.real();
   return a.imag() < b.imag();
}

vector<pt> convexHull(vector<pt> &v)
{
   sort(v.begin(), v.end(), cmp);
   /// remove duplicate points
   vector<pt> p;
   p.push_back(v[0]);

   for(int i = 1; i < v.size(); i++)
   {
       if(v[i] == v[i - 1])
           continue;
       p.push_back(v[i]);
   }

   if(p.size() <= 1)
       return p;

   v = p;
   for(int i = 1; i < (int) v.size(); i++)
   {
       if(v[i].imag() < v[0].imag())
           swap(v[i], v[0]);
       else if(dcmp(v[i].imag(), v[0].imag()) == 0 && v[i].real() <
v[0].real())
```

```
            swap(v[i], v[0]);
    }

    pivot = v[0];
    sort(v.begin() + 1, v.end(), sortCCW);

    //  ^^^ removes duplicate and sorts CCW according to bottom left point
^^^

    vector<pt> ch;
    v.push_back(v[0]);
    for(int i = 0; i < v.size(); i++)
    {
        while(ch.size() > 1)
        {
            auto cur = ch.back(), prv = ch[ch.size() - 2];

            if(cross(prv - cur, v[i] - cur) <= 0)
                break;
            ch.pop_back();
        }
        ch.push_back(v[i]);
    }
    v.pop_back();
    ch.pop_back();

    /// ch is convex hull with collinear v
//    return ch;
    /// removing collinear v

    vector<pt> finalCH = {ch[0]};
    for(int i = 1; i + 1 < ch.size(); i++)
        if(!isCollinear(ch[i - 1], ch[i], ch[i + 1]))
            finalCH.push_back(ch[i]);

    finalCH.push_back(ch.back());
    return finalCH;
}
```

Parametric line equation

```
pt p(ld t, pt &p0, pt &p1) // parametric equation for line p0 -> p1
{
    // t [0, 1]
```

```
    assert(t >= 0 && t <= 1);
    // if t > 1 after p1 (outside line)
    // if t < 0 before p0 (outside line)
    return (1 - t) * p0 + t * p1;
}
```

Is Point On Ray

```
bool pointOnRay(pt a, pt b, pt c)
{
    if(!isCollinear(a, b, c))
        return false;
    return dcmp(dot(b - a, c - a), 0) != -1;
}
```

Point to Line distance

```
long double pointLineDist(const point& a, const point& b, const point& p)
{
        if (same(a, b))
                return hypot(a.X - p.X, a.Y - p.Y);
// dist >= 0 := Point above line
// dist < 0 := Point Below line

        return fabs(cross(vec(a, b), vec(a, p)) / length(vec(a, b)));
}
```

Point to Segment distance

```
ld pointSegmentDist(const pt &a, const pt &b, const pt &p)
{
    ld len = abs(a - b) * abs(a - b);
    if(dcmp(len, 0) == 0)
        return abs(p - a);

    ld t = max((ld) 0, min((ld) 1, dot(p - a, b - a) / len));
    pt x = a + t * (b - a);
    return abs(x - p);
}
```

Line Lattice Points Count

```
int lineLatticePointsCount(int x1, int y1, int x2, int y2)
{
        return abs(__gcd(x1 - x2, y1 - y2)) + 1;
```

```
}
```

Line Line Intersection

```
pt lineLineIntersection(pt A, pt B, pt C, pt D)
{
    // Line AB represented as a1x + b1y = c1
    ld a1 = B.y - A.y;
    ld b1 = A.x - B.x;
    ld c1 = A.x * B.y - B.x * A.y;
    // Line CD represented as a2x + b2y = c2
    ld a2 = D.y - C.y;
    ld b2 = C.x - D.x;
    ld c2 = C.x * D.y - D.x * C.y;

    ld determinant = a1 * b2 - a2 * b1;
    if(determinant == 0)
    {
        // Same line aka Rakbin fo2 b3d
        if(isCollinear(A, B, C) && isCollinear(A, B, D))
            return pt(INF, INF);

        // The lines are parallel.
        return pt(-INF, -INF);
    }
    else
    {
        ld X = (b2 * c1 - b1 * c2) / determinant;
        ld Y = (a1 * c2 - a2 * c1) / determinant;
        return pt(X, Y);
    }
}
```

Circumcircle

```
pair<ld, pt> circumCircle(pt a, pt b, pt c)
{
    ld A1 = 2 * (b.real() - a.real());
    ld B1 = 2 * (b.imag() - a.imag());
    ld C1 = b.real() * b.real() + b.imag() * b.imag() - a.real() * a.real()
- a.imag() * a.imag();
    ld A2 = 2 * (c.real() - b.real());
    ld B2 = 2 * (c.imag() - b.imag());
    ld C2 = c.real() * c.real() + c.imag() * c.imag() - b.real() * b.real()
- b.imag() * b.imag();
```

```
    ld x = ((C1 * B2) - (C2 * B1)) / ((A1 * B2) - (A2 * B1));
    ld y = ((A1 * C2) - (A2 * C1)) / ((A1 * B2) - (A2 * B1));
    pt center(x, y);
    return make_pair(norm((center - A1)), center);
}
```

Circle Line Intersections

```
int circleLineIntersection(const point& p0, const point& p1, const point&
cen, long double rad, point& r1, point & r2)
{
    if (same(p0, p1))
    {
        if (fabs(lengthSqr(vec(p0, cen)) - (rad * rad)) < EPS)
        {
            r1 = r2 = p0;
            return 1;
        }
        return 0;
    }
    long double a, b, c, t1, t2;
    a = dot(p1 - p0, p1 - p0);
    b = 2 * dot(p1 - p0, p0 - cen);
    c = dot(p0 - cen, p0 - cen) - rad * rad;
    double det = b * b - 4 * a * c;
    int res;
    if (fabs(det) < EPS)
        det = 0, res = 1;
    else if (det < 0)
        res = 0;
    else
        res = 2;
    det = sqrt(det);
    t1 = (-b + det) / (2 * a);
    t2 = (-b - det) / (2 * a);
    r1 = p0 + t1 * (p1 - p0);
    r2 = p0 + t2 * (p1 - p0);
    return res;
}
```

CircleCircle Intersection

```
int circleCircleIntersection(pt c1, ld r1, pt c2, ld r2, pt &res1, pt
&res2)
{
   if(same(c1, c2) && fabs(r1 - r2) < eps)
   {
       res1 = res2 = c1;
       return fabs(r1) < eps ? 1 : INF;
   }
   ld len = abs(c2 - c1);
   if(fabs(len - (r1 + r2)) < eps || fabs(fabs(r1 - r2) - len) < eps)
   {
       pt d, c;
       ld r;
       if(r1 > r2)
           d = c2 - c1, c = c1, r = r1;
       else
           d = c1 - c2, c = c2, r = r2;
       res1 = res2 = (d / abs(d)) * r + c;
       return 1;
   }
   if(len > r1 + r2 || len < fabs(r1 - r2))
       return 0;
   ld a = cosRule(r2, r1, len);
   pt c1c2 = ((c2 - c1) / abs(c2 - c1)) * r1;
   res1 = rotateAbout(c1c2, pt(0, 0), a) + c1;
   res2 = rotateAbout(c1c2, pt(0, 0), -a) + c1;
   return 2;
}
```

Minimum enclosing circle MEC (untested)

```
pt pnts[100001], r[3], cen;
ld rad;
int ps, rs;    // ps = n, rs = 0, initially
// Pre-condition
// random_shuffle(pnts, pnts+ps);
void MEC()
{
   if(ps == 0 && rs == 2)
   {
       cen = (r[0] + r[1]) / (ld) 2.0;
       rad = length(r[0] - cen);
```

```
    }
    else if(rs == 3)
    {
        pair<ld, pt> p = circumCircle(r[0], r[1], r[2]);
        cen = p.second;
        rad = p.first;
    }
    else if(ps == 0)
    {
        cen = r[0];    // sometime be garbage, but will not affect
        rad = 0;
    }
    else
    {
        ps--;
        MEC();
        if(length(pnts[ps] - cen) > rad)
        {
            r[rs++] = pnts[ps];
            MEC();
            rs--;
        }
        ps++;
    }
}
```

Polygon centroid

```
pt polygonCentroid(vector<pt> &v)
{
    // can be implemented using pairs though to save time
    // cross product = (x1*y2) - (x2*y1)

    int lst = v.size() - 1;
    ld area = 0, X = 0, Y = 0, c;

    for(int i = 0; i < v.size(); i++)
    {
        c = cross(v[lst], v[i]), area += c;
        X += (v[i].real() + v[lst].real()) * c;
        Y += (v[i].imag() + v[lst].imag()) * c;

        lst = i;
    }

    if(area < eps) // line
        return (v[0] + v.back()) * (ld) 0.5;
```

```
    area /= 2;
    X /= 6.0 * area, Y /= 6.0 * area;

    // extra precision

    if(X < eps)
        X = 0;
    if(Y < eps)
        Y = 0;

    return pt(X, Y);
}
```

Point in polygon

```
bool isInsidePolygon(vector<pt> &v, pt p)
{
    int wn = 0;
    int cur = v.size() - 1;
    for(int nxt = 0; nxt < v.size(); nxt++)
    {
        if(pointOnSegment(v[cur], v[nxt], p))
            return false;

        if(v[cur].imag() <= p.imag()) // up
            wn += (v[nxt].imag() > p.imag() && cross(v[nxt] - v[cur], p -
v[cur]) > eps);
        else // down
            wn -= (v[nxt].imag() <= p.imag() && cross(v[nxt] - v[cur], p -
v[cur]) < -eps);
        cur = nxt;
    }
    return wn;
}
```

Point in CONVEX polygon multiple queries

```
// Build in NlogN and each query in LogN
#define X real()
#define Y imag()
#define dot(a, b) ((conj(a) * (b)).real())
#define cross(a, b) ((conj(a) * (b)).imag())
#define vec(a, b) (point)(b - a)
const ld eps = 1e-12;
```

```
const int PolygonCnt = 1 + 4;

int dcmp(const ld &a, const ld &b)
{
   if(fabs(a - b) < eps)
       return 0;

   return (a > b ? 1 : -1);
}

bool lexComp(const point &a, const point &b)
{
   return a.X < b.X or (dcmp(a.X, b.X) == 0 and a.Y < b.Y);
}

int sign(ld val)
{
   return (val > eps ? 1 : (val < -eps ? -1 : 0));
}

bool pointInTriangle(point a, point b, point c, point pt)
{
   ld s1 = abs(cross(vec(a, b), vec(a, c)));
   ld s2 = abs(cross(vec(pt, a), vec(pt, b))) + abs(cross(vec(pt, b),
vec(pt, c))) +
           abs(cross(vec(pt, c), vec(pt, a)));
   return dcmp(s1, s2) == 0;
}

bool isPointOnSegment(point a, point b, point c)
{
   ld acb = abs(a - b), ac = abs(a - c), cb = abs(b - c);
   return dcmp(acb - (ac + cb), 0) == 0;
}

vector<point> seq[PolygonCnt];
point translation[PolygonCnt], polygonStart[PolygonCnt],
polygonEnd[PolygonCnt];
int num[PolygonCnt];

/// todo: call pre
/// give polygon to answer queries on
void pre(vector<point> &points, int idx)
{
   num[idx] = points.size();
   int pos = 0;
```

```
    for(int i = 1; i < num[idx]; i++)
        if(lexComp(points[i], points[pos]))
            pos = i;

    rotate(points.begin(), points.begin() + pos, points.end());
    num[idx]--;
    seq[idx].resize(num[idx]);

    for(int i = 0; i < num[idx]; i++)
        seq[idx][i] = points[i + 1] - points[0];

    translation[idx] = points[0];

    polygonStart[idx] = points[0];
    polygonEnd[idx] = points.back();
}

bool pointInConvexPolygon(point pt, int idx)
{
    if(seq[idx].size() == 1)
        return isPointOnSegment(polygonStart[idx], polygonEnd[idx], pt);

    pt = pt - translation[idx];

    if(dcmp(cross(seq[idx][0], pt), 1) != 0 and
       sign(cross(seq[idx][0], pt)) != sign(cross(seq[idx][0],
seq[idx][num[idx] - 1])))
        return 0;

    if(dcmp(cross(seq[idx][num[idx] - 1], pt), 0) != 0 and
       sign(cross(seq[idx][num[idx] - 1], pt)) !=
sign(cross(seq[idx][num[idx] - 1], seq[idx][0])))
        return 0;

    if(dcmp(cross(seq[idx][0], pt), 0) == 0)
        return dcmp(dot(seq[idx][0], seq[idx][0]), dot(pt, pt)) != -1;

    int l = 0, r = num[idx] - 1;
    while(r - l > 1)
    {
        int mid = (l + r) / 2;
        int pos = mid;

        if(dcmp(cross(seq[idx][pos], pt), 0) != -1)
            l = mid;
        else
            r = mid;
```

```
    }

    int pos = l;
    return pointInTriangle(seq[idx][pos], seq[idx][pos + 1], point(0, 0),
pt);
}
bool pointOnPolygon(vector<point> &p, point pt)
{
    int sz = p.size();
    int s = 1, e = sz - 1, m, before = -1, after = -1;
    while(s <= e)
    {
        m = (s + e) >> 1;
        if(cross(vec(p[0], p[m]), vec(p[0], pt)) >= 0)
            before = m, s = m + 1;
        else
            e = m - 1;
    }

    s = 1, e = sz - 1;
    while(s <= e)
    {
        m = (s + e) >> 1;

        if(cross(vec(p[0], pt), vec(p[0], p[m])) >= 0)
            after = m, e = m - 1;
        else
            s = m + 1;
    }

    // if before == after if(1 or n - 1) check seg else check point
    if(before == -1 or after == -1)
        return 0;

    if(before == after)
    {
        if(before == 1 or before == sz - 1)
            return isPointOnSegment(p[0], p[before], pt);
        else
            return abs(p[before] - pt) < 1e-12;
    }

    return isPointOnSegment(p[before], p[after], pt);
}
```

Pick's Theorem (Count number of internal integer points)

```
ll picksTheorm(vector<pt> &p)
{
   ld area = 0;
   int bound = 0;

   p.push_back(p[0]);
   for(int i = 0; i < p.size() - 1; i++)
   {
       int j = i + 1;
       area += cross(p[i], p[j]);
       pt v = p[j] - p[i];
       bound += abs(__gcd((int) v.real(), (int) v.imag()));
   }
   p.pop_back();

   area /= 2.0;
   area = fabs(area);
   return round(area - bound / 2 + 1);
}
```

Area of union rectangles

```
class Rectangle{
public:
    int x1 , y1 , x2 , y2;
    static Rectangle empt;
    Rectangle(){
        x1 = y1 = x2 = y2 = 0;
    }
    Rectangle(int X1, int Y1, int X2, int Y2)
    {
        x1 = X1; y1 = Y1;
        x2 = X2; y2 = Y2;
    }
    Rectangle intersect(Rectangle R)
    {
        if (R.x1 >= x2 || R.x2 <= x1)
            return empt;
        if (R.y1 >= y2 || R.y2 <= y1)
            return empt;
        return Rectangle(max(x1 , R.x1) , max(y1 , R.y1) , min(x2 ,
R.x2) , min(y2 , R.y2));
```

```
    }
};
struct Event{
    int x , y1 , y2 , type;
    Event(){}
    Event(int x , int y1 , int y2 , int type):x(x) , y1(y1) , y2(y2) ,
type(type){}
};
bool operator < (const Event&A , const Event&B){
    //if(A.x != B.x)
    return A.x < B.x;
    //if(A.y1 != B.y1) return A.y1 < B.y1;
    //if(A.y2 != B.y2()) A.y2 < B.y2;
}
const int MX=(1<<17);
struct Node{
    int prob , sum , ans;
    Node(){}
    Node(int prob , int sum , int ans):prob(prob) , sum(sum) ,
ans(ans){}
};
Node tree[MX*4];
int interval[MX];
void build(int x , int a , int b){
    tree[x] = Node(0 , 0 , 0);
    if(a==b){
        tree[x].sum+=interval[a];
        return;
    }
    build(x*2 , a , (a+b)/2);
    build(x*2+1 , (a+b)/2+1 , b);
    tree[x].sum = tree[x*2].sum + tree[x*2+1].sum;
}
int ask(int x){
    if(tree[x].prob) return tree[x].sum;
    return tree[x].ans;
}
int st , en , V;
void update(int x , int a , int b){
    if(st>b || en<a) return;
    if(a>=st && b<=en){
        tree[x].prob+=V;
        return;
    }
    update(x*2 , a , (a+b)/2);
    update(x*2+1 , (a+b)/2+1 , b);
    tree[x].ans = ask(x*2) + ask(x*2+1);
```

```
}
Rectangle Rectangle::empt = Rectangle();
vector < Rectangle > Rect;
vector < int > sorted;
vector < Event > sweep;
void compressncalc(){
    sweep.clear();
    sorted.clear();
    for(auto R : Rect){
        sorted.push_back(R.y1);
        sorted.push_back(R.y2);
    }
    sort(sorted.begin() , sorted.end());
    sorted.erase(unique(sorted.begin() , sorted.end()) , sorted.end());
    int sz = sorted.size();
    for(int j=0;j<sorted.size() - 1;j++)
        interval[j+1] = sorted[j+1] - sorted[j];
    for(auto R : Rect){
        sweep.push_back(Event(R.x1 , R.y1 , R.y2 , 1));
        sweep.push_back(Event(R.x2 , R.y1 , R.y2 , -1));
    }
    sort(sweep.begin() , sweep.end());
    build(1,1,sz-1);
}
long long ans;
void Sweep(){
    ans=0;
    if(sorted.empty() || sweep.empty()) return;
    int last = 0 , sz_ = sorted.size();
    for(int j=0;j<sweep.size();j++){
        ans+= 1ll * (sweep[j].x - last) * ask(1);
        last = sweep[j].x;
        V = sweep[j].type;
        st = lower_bound(sorted.begin() , sorted.end() , sweep[j].y1) -
sorted.begin() + 1;
        en = lower_bound(sorted.begin() , sorted.end() , sweep[j].y2) -
sorted.begin();
        update(1 , 1 , sz_-1);
    }
}
int main()
{
    int n;
    scanf("%d", &n);
    for(int j = 1; j <= n; j++)
    {
        int a , b ,c  , d;
```

```
        scanf("%d %d %d %d",&a,&b,&c,&d);
        Rect.push_back(Rectangle(a , b , c , d));
    }
    compressncalc();
    Sweep();
    cout<<ans<<endl;
}
```

Algebra

Miller Rabin

```
using u128 = __uint128_t;

/// return (s * m) % mod
 ll mult(ll s, ll m, ll mod){
      if(!m) return 0;
      ll ret = mult(s, m/2, mod);
      ret = (ret + ret) % mod;
      if(m & 1) ret = (ret + s) % mod;
      return ret;
}

ll fp(ll b, ll p, ll mod){
      ll res = 1;
      b %= mod;
      while(p > 0){
      if(p & 1) res = (u128) res * b % mod;
      b = (u128) b * b % mod;
      p >>= 1;
      }
      return res;

}

bool check(ll n, ll a, ll d, ll s){
      ll x = fp(a, d, n);
      if(x == 1 || x == n - 1) return 0;
      for(int r = 1; r < s; r++){
      x = (u128) x * x % n;
      if(x == n - 1) return 0;
      }
      return 1;
}

bool MillerRabin(ll n){
      if(n < 2) return 0;

      int r = 0;
      ll d = n - 1;
      while((d & 1) == 0){
```

```
        d >>= 1;
        r++;
        }
        for(ll a: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}){
        if(n == a) return 1;
        if(check(n, a, d, r)) return 0;
        }
        return 1;
}
```

GCD

```
ll gcd(ll a , ll b)
{
    if(b == 0)
        return a;
    return gcd(b, a % b);
}
```

Extended GCD

```
int eGCD(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;

        return a;
    }

    int newX, newY;
    int d = eGCD(b, a % b, newX, newY);

    x = newY;
    y = newX - a / b * newY;

    return d;
}
```

Fast Power

```
int modPow(int b, int p) {
    if (p == 0)
```

```
        return 1;

    int x = modPow(b, p >> 1);

    return p % 2 == 0 ? mul(x, x) : mul(b, mul(x, x));
}
```

Sieve

```
vector<bool> prime (n + 1, true);

void sieve(int n)
{
    prime[0] = prime[1] = false;
    for (int i = 2; i <= n; i++)
        for (int j = i * i; j <= n && prime[i] && i * 1ll * i <= n; j += i)
            prime[j] = false;
}
```

Fast Fibonacci

```
int fast_Fibonacci(int n)
{
    int i = 1, h = 1, j = 0, k = 0, t;
    while (n > 0)
    {
        if (n % 2 == 1)
            t = j * h, j = i * h + j * k + t, i = i * k + t;
        t = h * h, h = 2 * k * h + t, k = k * k + t, n = n / 2;
    }
    return j;
}
```

Mod Inverse (using eGCD)

```
int modInv(int c, int M) {
   int x, y;

   int g = eGCD(c, M, x, y);

   if (g != 1) return -1;

   return (x + M) % M;
}
```

Mod Inverse (using Fast Power)

```
int modInvFer(int n) {
   return modPow(n, M - 2);
}
```

Pre Inverse

```
void pre()
{
    fact[0] = inv[0] = 1;
    for(int i = 1; i < N; i++)
        fact[i] = mul(fact[i - 1], i);

    inv[N - 1] = fp(fact[N - 1], mod - 2);
    for(int i = N - 2; i >= 1; i--)
        inv[i] = mul(inv[i + 1], i + 1);
}
```

Matrix Multiplication

```
matrix mul(matrix &a, matrix &b)//b and a can be replaced just check assert
{
   int n=a.size(),k=a[0].size();
   int k2=b.size(),m=b[0].size();
   assert(k == k2);
   matrix ans(n, vector<ll> (m, 0));
   for (int i=0;i<n;i++)
   {
       for (int j=0;j<m;j++)
       {
           for (int l=0;l<k2;l++)
           {
               ans[i][j]=add(ans[i][j],mul(a[i][l],b[l][j]));
           }
       }
   }
   return ans;
}
```

Permutations

```
int ncr(int n, int r) {
   return mul(fact[n], mul(modInvFer(fact[r]), modInvFer(fact[n - r])));
}


int npr(int n, int r)
{
    return mul(fact[n] * modInvFer(fact[n - r])) % mod;
}
```

Matrix inverse

```
void mat_inv()
{
    for(int i = 0; i < n; ++i)
        for(int j = 0; j < n + 1; ++j)
            cin >> mat[i][j];
    for(int i = 0; i < n; ++i)
        for(int j = 0; j < 2 * n + 1; ++j)
            if(j == (i + n + 1))
                mat[i][j] = 1;
    for(int i = n; i > 1; --i)
        if(mat[i-1][1] < mat[i][1])
            for(int j = 0; j < 2 * n + 1; ++j)
            {
                ld d = mat[i][j];
                mat[i][j] = mat[i - 1][j];
                Mat[i - 1][j] = d;
            }
    for(int i = 0; i < n; ++i)
        for(int j = 0; j < 2 * n + 1; ++j)
            if(j != i)
            {
                ld d = mat[j][i] / mat[i][i];
                for(int k = 0; k < n * 2 + 1; ++k)
                    mat[j][k] -= mat[i][k] * d;
            }
    for(int i = 0; i < n; ++i) {
        ld d = mat[i][i];
        for(int j = 0; j < 2 * n + 1; ++j)
            mat[i][j] = mat[i][j] / d;
    }
```

```
    vector<ld> eq_ans;
    for(int i = 0; i < n; ++i)
    {
        for(int j = n; j < 2 * n + 1; ++j)
            if(j == n)
                eq_ans.push_back(mat[i][j]);
            else
                cout << mat[i][j] << " ";
        cout << endl;
    }
    for(auto i : eq_ans)
        cout<<i<<endl;
}
```

Find Any Solution

```
bool findAnySolution(int a, int b, int c, int &x, int &y, int &g) {
    g = eGCD(abs(a), abs(b), x, y);

    if (c % g)
        return 0;

    x *= c / g;
    y *= c / g;

    if (a < 0)
        x *= -1;
    if (b < 0)
        y *= -1;

    return 1;
}
```

Shifting Solution

```
void shiftSolution(int &x, int &y, int a, int b, int cnt) {
    x += cnt * b;
    y -= cnt * a;
}
```

Find All Solutions

```
int findAllSolutions(int a, int b, int c, int mnX, int mxX, int mnY, int
mxY) {
    int x, y, g;

    if (!findAnySolution(a, b, c, x, y, g))
        return 0;

    a /= g;
    b /= g;

    int signA = a > 0 ? 1 : -1;
    int signB = b > 0 ? 1 : -1;

    shiftSolution(x, y, a, b, (mnX - x) / b);

    if (x < mnX)
        shiftSolution(x, y, a, b, signB);

    if (x > mxX)
        return 0;

    int lx1 = x;

    shiftSolution(x, y, a, b, (mxX - x) / b);

    if (x > mxX)
        shiftSolution(x, y, a, b, -signB);

    int rx1 = x;

    shiftSolution(x, y, a, b, -(mnY - y) / a);

    if (y < mnY)
        shiftSolution(x, y, a, b, -signA);

    if (y > mxY)
        return 0;

    int lx2 = x;

    shiftSolution(x, y, a, b, -(mxY - y) / a);

    if (y > mxY)
        shiftSolution(x, y, a, b, signA);

    int rx2 = x;
```

```
    if (lx2 > rx2)
        swap(lx2, rx2);

    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

    if (lx > rx)
        return 0;

    return (rx - lx) / abs(b) + 1;
}
```

CRT

```
ll CRT(vector<ll> &rems, vector<ll> &mods){
    ll prevRem = rems[0], prevMod = mods[0]; /// first congruence

    for(int i = 1; i < rems.size(); i++){
        ll x, y, c = rems[i] - prevRem;

        if(c % __gcd(prevMod, -mods[i])) /// LDE can't be solved (no
answer to system of congruences)
            return -1;

        ll g = eGCD(prevMod, -mods[i], x, y);
        x *= c / g;

        prevRem += prevMod * x;
        prevMod = prevMod / g * mods[i];
        prevRem = ((prevRem % prevMod) + prevMod) % prevMod;
    }

    return prevRem;
}
```

Phi Sieve

```
int phi[N];

void sieve() {
   for (int i = 0; i < N; i++)
       phi[i] = i;

   phi[1] = 1;

   for (int i = 2; i < N; i++) {
       if (phi[i] == i) {
           for (int j = i; j < N; j += i)
               phi[j] -= phi[j] / i;
       }
   }
}
```

Phi

```
int phi(int x){
    int ret = x;

    for(int i = 2; i * i <= x; i++){
        if(x % i == 0){
            while(x % i == 0)
                x /= i;
            ret -= ret / i;
        }
    }

    if(x > 1)
        ret -= ret / x;

    return ret;
}
```

Mobius Sieve

```
int mobius[N];
bool isPrime[N];

void mobiusSieve(){
    mobius[1] = 1;
    for(int i = 2; i < N; i++)
        mobius[i] = 1, isPrime[i] = 1;

    for(int i = 2; i < N; i++){
        if(isPrime[i]){
            mobius[i] = -1;
            for(int j = 2 * i; j < N; j += i){
                mobius[j] = (j % (i * i) == 0 ? 0 : -mobius[j]);
                isPrime[j] = 0;
            }
        }
    }
}
```

Mobius

```
int mobius(int x){
    int ret = 1;

    for(int i = 2; i * i <= x; i++){
        if(x % i == 0){
            if(x % (i * i) == 0)
                return 0;

            x /= i, ret *= -1;
        }
    }

    if(x > 1)
        ret *= - 1;

    return ret;
}
```

Burnside Lemma

$$|\text{Classes}| = \frac{1}{|G|} \sum_{\pi \in G} k^{C(\pi)}$$

```
int n, m, c;
cin >> n >> m >> c;

int ways = modPow(c, n * n);

int sol = 0;
for (int i = 0; i < m; i++) {
   int g = __gcd(i, m);

   sol = add(sol, modPow(ways, g));
}

cout << mul(sol, modInvFer(m));
```

Catalan Numbers

```
const int MOD = ....
const int MAX = ....

int catalan[MAX];

void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1])% MOD;
            if (catalan[i] >= MOD)
                catalan[i] -= MOD;
        }
    }
}
```

```
// 1- Number of correct bracket sequence consisting of n opening and n
closing brackets.

// 2- The number of rooted full binary trees with n+1 leaves (vertices
are not numbered).

// A rooted binary tree is full if every vertex has
either two children or no children.

// 3- The number of ways to completely parenthesize n+1 factors.

// 4- The number of triangulations of a convex polygon with n+2 sides

// (i.e. the number of partitions of polygon into disjoint triangles by
using the diagonals).

// 5- The number of ways to connect the 2n points on a circle to form n
disjoint chords.

// 6- The number of non-isomorphic full binary trees with n internal
nodes (i.e. nodes having at least one son).

// 7- The number of monotonic lattice paths from point (0,0) to point
(n,n) in a square lattice of size nxn,

// which do not pass above the main diagonal (i.e. connecting (0,0) to
(n,n)).

// 8- Number of permutations of length n that can be stack sorted

// (i.e. it can be shown that the rearrangement is stack sorted if and
only if there is no such index i<j<k, such that ak<ai<aj).

// 9- The number of non-crossing partitions of a set of n elements.

// 10- The number of ways to cover the ladder 1..n using n rectangles
```

Pascals Triangle

1																
1	1															
1	2	1														
1	3	3	1													
1	4	6	4	1												
1	5	10	10	5	1											
1	6	15	20	15	6	1										
1	7	21	35	35	21	7	1									
1	8	28	56	70	56	28	8	1								
1	9	36	84	126	126	84	36	9	1							
1	10	45	120	210	252	210	120	45	10	1						
1	11	55	165	330	462	462	330	165	55	11	1					
1	12	66	220	495	792	924	792	495	220	66	12	1				
1	13	78	286	715	1287	1716	1716	1287	715	286	78	13	1			
1	14	91	364	1001	2002	3003	3432	3003	2002	1001	364	91	14	1		
1	15	105	455	1365	3003	5005	6435	6435	5005	3003	1365	455	105	15	1	
1	16	120	560	1820	4368	8008	11440	12870	11440	8008	4368	1820	560	120	16	1

Prime numbers

	2	3	5	7	11	13	17	19	23
29	31	37	41	43	47	53	59	61	67
71	73	79	83	89	97	101	103	107	109
113	127	131	137	139	149	151	157	163	167
173	179	181	191	193	197	199	211	223	227
229	233	239	241	251	257	263	269	271	277
281	283	293	307	311	313	317	331	337	347
349	353	359	367	373	379	383	389	397	401
409	419	421	431	433	439	443	449	457	461
463	467	479	487	491	499	503	509	521	523
541	547	557	563	569	571	577	587	593	599
601	607	613	617	619	631	641	643	647	653
659	661	673	677	683	691	701	709	719	727
733	739	743	751	757	761	769	773	787	797
809	811	821	823	827	829	839	853	857	859
863	877	881	883	887	907	911	919	929	937
941	947	953	967	971	977	983	991	997	

Graphs

K-th Shortest path

```
struct edge
{
    int s, e, c; //start, end, cost
    bool operator <(const edge& e)const
    {
        return c < e.c;
    }
};
const int SIZE = 100; //max nodes number
int N, start, end, K; //find the k-th shortest path from start to end
int dist[SIZE][SIZE]; //this can be adjList instead of adjMatrix
//Returns -1 of no k-th shortest path exist between start and end
int getKthShortestPath()
{
    multiset<edge> pq; //first is cost and second is node
    edge e = {-1, start, 0};
```

```
    pq.insert(e);
    vector<int> reached[N]; //reached[i][j] is the cost of the j-th
shortest path from start to i
    while(!pq.empty())
    {
        edge e = *pq.begin();
        pq.erase(pq.begin());
        if(reached[e.e].size() >= K)
            continue;
        reached[e.e].push_back(e.c);
        for(int i = 0 ; i < N ; i++)
        {
            if(dist[e.e][i] == -1)
                continue;
            edge ne = {e.e, i, e.c + dist[e.e][i]};
            pq.insert(ne);
        }
    }
    //no k-th path exist between start and end 23
    return reached[end].size() >= K? reached[end].back(): -1;
}
//MAIN
memset(dist, -1, sizeof dist);
//set N, start, end, K, dist
```

Floyd

```
void printPath(int path[][N], int v, int u)
{
      if (path[v][u] == v)
            return;
      printPath(path, v, path[v][u]);
      cout << path[v][u] << " ";
}

// Function to run Floyd-Warshell algorithm
void FloydWarshell(int adjMatrix[][N])
{
      // cost[] and parent[] stores shortest-path
      // (shortest-cost/shortest route) information
      int cost[N][N], path[N][N];

      // initialize cost[] and parent[]
      for (int v = 0; v < N; v++)
      {
            for (int u = 0; u < N; u++)
            {
```

```
                // initally cost would be same as weight
                // of the edge
                cost[v][u] = adjMatrix[v][u];

                if (v == u)
                      path[v][u] = 0;
                else if (cost[v][u] != INT_MAX)
                      path[v][u] = v;
                else
                      path[v][u] = -1;
            }
      }
      // run Floyd-Warshell
      for (int k = 0; k < N; k++)
      {
            for (int v = 0; v < N; v++)
            {
                for (int u = 0; u < N; u++)
                {
                      // If vertex k is on the shortest path from v to u,
                      // then update the value of cost[v][u], path[v][u]

                      if (cost[v][k] != INT_MAX && cost[k][u] != INT_MAX
                            && cost[v][k] + cost[k][u] < cost[v][u])
                      {
                            cost[v][u] = cost[v][k] + cost[k][u];
                            path[v][u] = path[k][u];
                      }
                }
                // if diagonal elements become negative, the
                // graph contains a negative weight cycle
                if (cost[v][v] < 0)
                {
                      cout << "Negative Weight Cycle Found!!";
                      return;
                }
            }
      }
      // Print the shortest path between all pairs of vertices
      printSolution(cost, path);
}
```

```
//To check if dist[a][b] = -inf ba3d ma a3ml floyd mara
for(int k = 0; k < n; k ++)
    for(int i = 0; i < n; i ++)
```

```
        for(int j = 0; j < n; j ++)
            if(dist[i][k] < inf && dist[k][j] < inf && dist[k][k] < 0)
               dist[i][j] = -inf;
```

Tarjan

```
vector< vector<int> > adjList, comps;
vector<int> inStack, lowLink, dfn, comp;
stack<int> stk;
int ndfn;

void tarjan(int node)
{
    lowLink[node] = dfn[node] = ndfn++, inStack[node] = 1;
    stk.push(node);
    for(int i = 0; i < adjList[node].size(); i++)
    {
        int ch = adjList[node][i];
        if (dfn[ch] == -1)
        {
            tarjan(ch);
            lowLink[node] = min(lowLink[node], lowLink[ch]);
        }
        else if (inStack[ch])
            lowLink[node] = min(lowLink[node], dfn[ch]);
    }
    if (lowLink[node] == dfn[node])
    {
        comps.push_back(vector<int> ());
        int x = -1;
        while (x != node)
        {
            x = stk.top(), stk.pop(), inStack[x] = 0;
            comps.back().push_back(x);
            comp[x] = comps.size() - 1;
        }
    }
}

void scc()
{
    int n = sz(adjList);
```

```
    inStack.clear(), inStack.resize(n), lowLink.clear(), lowLink.resize(n);
    dfn.clear(), dfn.resize(n, -1), ndfn = 0;
    comp.clear(), comp.resize(n);
    comps.clear();
    for(int i = 0; i < n; i++)
        if (dfn[i] == -1)
            tarjan(i);
}
```

Prim

```
int cost[NSize][NSize], d[NSize];
bool V[NSize];
const int inf = INT_MAX;
// take care of multiple edges
void mstPrim(int start, int n)
{
    // lazem undirected graph
    for(int i = 0; i < n; i++)
        d[i] = inf; // take care of over flow when sum
    memset(V, 0, sizeof V);
    d[start] = 0;
    int current = start;
    int minD = inf;
    int nextInd = -1; // minmum to choose
    while(current != -1)
    {
        V[current] = 1;
        minD = inf;
        nextInd = -1;
        for(int i = 0; i < n; ++i)
        {
            if(!V[i] && cost[current][i] < d[i])
            {
               d[i] = cost[current][i];
            // law 3awez ageb el path we ana b relax b5zn el parent
            // parent[i] = current;
            }
            if(!V[i] && d[i] < minD)
            {
                nextInd = i;   // b3ml mimization bdal el for loop el zyada
                minD = d[i];
            }
        }
        current = nextInd;
    }
}
```

Centroid

```
void dfs(int u, int p)
{
    sz[u] = 1;
    for (int i = 0; i < vc[u].size(); i++)
    {
        int v = vc[u][i];
        if (v != p)
        {
            dfs(v, u);
            sz[u] += sz[v];
        }
    }
}

int findcentroid(int u)
{
    int p = -1;
    dfs(u, -1);
    int cap = sz[u] >> 1;
    while (1)
    {
        bool found = false;
        for (int i = 0; i < vc[u].size(); i++)
        {
            int v = vc[u][i];
            if (v != p && sz[v] > cap)
            {
                found = true;
                p = u;
                u = v;
                break;
            }
        }
        if (!found)
            return u;
    }
}
```

Check Odd Cycle

```
bool CheckOddCycle()
{
	int vis[N + 2];
	memset(vis, -1, sizeof vis);
	vis[1] = 1;
	queue<int> q;
	q.push(1);
	while (!q.empty())
	{
		int node = q.front();
		q.pop();
		for (auto it: v[node])
		{
			if (vis[it] == -1)
			{
				vis[it] = 1 - vis[node];
				q.push(it);
			}
			else if (vis[it] == vis[node])
				return true;
		}
	}
	return false;
}
```

Euler Path

```
// O(V + E)
// Euler path is a path that uses every edge in graph exactly once
int n, in[N], out[N]; // in and out degrees
vector<int> path; // has the euler path in reverse order
queue<int> adj[N];

void dfs(int u)
{
   while(adj[u].size())
   {
      int v = adj[u].front();
      adj[u].pop();
      dfs(v);
```

```
    }
    path.push_back(u);
}

pair<bool, int> is_eulerian() // {is eulerian, start node of dfs}
{
    int _s = 0, _e = 0, st = 1;
    for(int u = 1; u < N; u++)
    {
        int sub = in[u] - out[u];
        if(sub == -1)
            ++_s, st = u;
        else if(sub == 1)
            ++_e;
        else if(sub != 0)
        return {0, -1};
    }
    if(_s > 1 || _e > 1 || _s + _e == 1)
        return {0, -1};
    return {1, st};
}
```

Centroid Decomposition

```
int sz[N];
bool vis[N];
vector<int> G[N];

int dfsSZ(int node, int par = -1)
{
    sz[node] = 1;
    for(auto &ch: G[node])
    {
        if(ch == par || vis[node])
            continue;
        sz[node] += dfsSZ(ch, node);
    }
    return sz[node];
}
```

```
int getCentroid(int node, int par, int &n)
{
   for(auto &ch: G[node])
   {
       if(ch == par || vis[ch])
           continue;

       if(sz[ch] << 1 > n)
           return getCentroid(ch, node, n);
   }
   return node;
}

void solveSubTree(int centroid, int par)
{

}

void decompose(int node, int par)
{
   dfsSZ(node, par);
   int centroid = getCentroid(node, par, sz[node]);

   vis[centroid] = true;
   solveSubTree(centroid, par);
   for(auto &ch: G[centroid])
   {
       if(ch == par || vis[ch])
           continue;
       decompose(ch, centroid);
   }
}
```

Finding a negative cycle in the graph

```
struct Edge {
    int a, b, cost;
};

int n, m;
vector<Edge> edges;
const int INF = 1000000000;

void solve()
{
    vector<int> d(n), p(n, -1);
    int x;
    for (int i = 0; i < n; ++i)
    {
        x = -1;
        for (Edge e: edges)
            if (d[e.a] + e.cost < d[e.b])
            {
                d[e.b] = d[e.a] + e.cost;
                p[e.b] = e.a;
                x = e.b;
        }
    }
    if (x == -1)
        cout << "No negative cycle found.";
    else
    {
        for (int i = 0; i < n; ++i)
            x = p[x];
        vector<int> cycle;
        for (int v = x;; v = p[v])
        {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1)
                break;
        }
        reverse(cycle.begin(), cycle.end());
        cout << "Negative cycle: ";
        for (int v: cycle)
            cout << v << ' ';
        cout << endl;
    }
}
```

DSU on trees

```
int dfs(int node, int p)
{
    int sz = 1, mx = -1;
    for(auto i: v[node])
        if(i != p)
        {
            int x = dfs(i, node);
            sz += x;
            if(x > mx)
                mx = x, heavy[node] = i;
        }
    return sz;
}

void add(int node, int p, int val )
{
    cnt[col[node]] += val;
    for(auto i: v[node])
        if(i != p)
            add(i, node, val);
}

void solve(int node, int p, int keep)
{
    for(auto i: v[node])
        if(i != p && i != heavy[node])
            solve(i, node, 0);
    if(heavy[node])
        ans=solve(heavy[node], node, 1);
    for(auto i: v[node])
        if(i != p && i != heavy[node])
            add(i, node, 1);
    cnt[col[node]]++;
    if(keep == 0){
        for(auto i: v[node])
            if(i != p)
                add(i, node, -1);
        cnt[col[node]]--;
    }
}
```

LCA

```
struct LCA
{
   int lvl[N];
   int up[N][LOG];

   LCA()
   {
       dfs(1, -1);
   }

   void dfs(int node, int par)
   {
       for(int i = 1; i < LOG; i++)
           up[node][i] = up[up[node][i - 1]][i - 1];

       for(auto &ch: G[node])
       {
           if(ch == par)
               continue;

           up[ch][0] = node;
           lvl[ch] = lvl[node] + 1;

           dfs(ch, node);
       }
   }

   int getKthParent(int node, int k)
   {
       for(int i = LOG - 1; i >= 0; i--)
       {
           if((k >> i) & 1)
               node = up[node][i];
       }
       return node;
   }

   int lca(int a, int b)
   {
       if(lvl[a] < lvl[b])
           swap(a, b);

       a = getKthParent(a, lvl[a] - lvl[b]);
```

```
        if(a == b)
            return a;

        for(int i = LOG - 1; i >= 0; i--)
        {
            if(up[a][i] != up[b][i])
                a = up[a][i], b = up[b][i];
        }

        return up[a][0];
    }

    int getMax(int a, int lca)
    {
        int ret = INT_MIN;
        int k = lvl[a] - lvl[lca];
        for(int i = LOG - 1; i >= 0; i--)
        {
            if((k >> i) & 1)
                ret = max(ret, mx[a][i]), a = up[a][i];
        }
        return ret;
    }
};
```

LCA (O(N) processing, O(1) query)

```
struct SparseTable {
    vector<int> p;
    vector<int> pre;
    vector<vector<int>> g;

    void build(vector<int> &v) {
        g.resize(N);
        pre.resize(N);
        for (int i = 0; i < N; i++)
            g[i].resize(LOG);

        p = v;

        int n = v.size();

        pre[1] = 0;
        for (int i = 2; i <= n; i++)
```

```
            pre[i] = pre[i >> 1] + 1;

        for (int i = 0; i < n; i++)
            g[i][0] = i;

        for (int k = 1; k < LOG; k++) {
            for (int i = 0; i + (1 << k) - 1 < n; i++) {
                if (v[g[i][k - 1]] <= v[g[i + (1 << (k - 1))][k - 1]])
                    g[i][k] = g[i][k - 1];
                else
                    g[i][k] = g[i + (1 << (k - 1))][k - 1];
            }
        }
    }

    int query(int l, int r) {
        int d = r - l + 1, k = pre[d];

        if (p[g[l][k]] <= p[g[r - (1 << k) + 1][k]])
            return g[l][k];

        return g[r - (1 << k) + 1][k];
    }
} st;

int n, t;
int up[N][LOG];
vector<int> g[N];
vector<int> d(N), in(N), out(N), lvl, tree;

void dfs(int u, int p) {
    in[u] = t++;

    up[u][0] = p;

    d[u] = d[p] + 1;

    for (int k = 1; k < LOG; k++)
        up[u][k] = up[up[u][k - 1]][k - 1];

    tree.push_back(u);
    lvl.push_back(d[u]);

    for (auto v: g[u]) {
        if (v != p) {
            dfs(v, u);
```

```
            tree.push_back(u);
            lvl.push_back(d[u]);
        }
    }

    out[u] = t++;
}

bool isAncestor(int u, int v) {
    return in[v] >= in[u] && out[v] <= out[u];
}

int lca(int u, int v) {
    if (isAncestor(u, v))
        return u;

    if (isAncestor(v, u))
        return v;

    return tree[st.query(min(in[u], in[v]), max(out[u], out[v]))];
}
```

Second Best Mst

```
const int N = 1e5 + 5, LOG = 18;

int lvl[N];
int up[N][LOG];
int mx[N][LOG];
bool usedInMst[N]; //EDGES SIZE RKZ :|
vector<pair<int, int>> G[N];

void dfs(int node, int par)
{
    for(int i = 1; i < LOG; i++)
    {
        up[node][i] = up[up[node][i - 1]][i - 1];
        mx[node][i] = max(mx[node][i - 1], mx[up[node][i - 1]][i - 1]);
    }

    for(auto ch: G[node])
    {
        if(ch.first == par)
            continue;
```

```
        up[ch.first][0] = node;
        mx[ch.first][0] = ch.second;
        lvl[ch.first] = lvl[node] + 1;
        dfs(ch.first, node);
    }
}

int getKthParent(int node, int k)
{
    for(int i = LOG - 1; i >= 0; i--)
    {
        if((k >> i) & 1)
            node = up[node][i];
    }
    return node;
}

int LCA(int a, int b)
{
    if(lvl[a] < lvl[b])
        swap(a, b);

    a = getKthParent(a, lvl[a] - lvl[b]);

    if(a == b)
        return a;

    for(int i = LOG - 1; i >= 0; i--)
    {
        if(up[a][i] != up[b][i])
            a = up[a][i], b = up[b][i];
    }
    return up[a][0];
}

int getMax(int a, int b)
{
    int ret = INT_MIN;

    int k = lvl[a] - lvl[b];
    for(int i = LOG - 1; i >= 0; i--)
    {
        if((k >> i) & 1)
            ret = max(ret, mx[a][i]), a = up[a][i];
    }
    return ret;
}
```

```
struct edge
{
   int u, v, w, id;

   edge(int u, int v, int w, int id) : u(u), v(v), w(w), id(id)
   {}

   bool operator<(const edge &other) const
   {
       return w < other.w;
   }
};

struct DSU
{
   int par[N], sz[N];

   void init(int n)
   {
       for(int i = 0; i <= n; i++)
           par[i] = i, sz[i] = 1;
   }

   int findSet(int a)
   {
       return (par[a] == a ? a : par[a] = findSet(par[a]));
   }

   void unionSet(int a, int b)
   {
       a = findSet(a), b = findSet(b);
       if(a == b)
           return;
       if(sz[b] > sz[a])
           swap(a, b);

       par[b] = a;
       sz[a] += sz[b];
   }

} dsu;

int kruskal(vector<edge> &edges)
{
   int a, b, ret = 0;
   sort(edges.begin(), edges.end());
```

```
    for(auto it: edges)
    {
        a = dsu.findSet(it.u), b = dsu.findSet(it.v);
        if(a != b)
        {
            ret += it.w;
            dsu.unionSet(a, b);
            usedInMst[it.id] = true;
            G[it.u].push_back({it.v, it.w});
            G[it.v].push_back({it.u, it.w});
        }
    }
    return ret;
}

int main()
{
    int n, m;
    cin >> n >> m;
    vector<edge> edges;
    for(int i = 0, u, v, w; i < m; i++)
        cin >> u >> v >> w, edges.emplace_back(edge(u, v, w, i));

    dsu.init(n);
    int mst = kruskal(edges); // also fills the graph with the mst

    up[1][0] = 1;
    mx[1][0] = INT_MIN;
    dfs(1, 1);

    int lca, val, u, v;
    int secondMst = INT_MAX;
    for(auto &edge: edges) // try all edges
    {
        if(usedInMst[edge.id])
            continue;

        u = edge.u, v = edge.v;
        lca = LCA(u, v);
        val = mst - max(getMax(edge.u, lca), getMax(edge.v, lca)) +
edge.w;
        if(val > mst)
            secondMst = min(secondMst, val);
    }
    cout << secondMst;
}
```

Bellman-ford

```
struct edge
{
    int a, b, cost;
};

int n, m, v;
vector<edge> e;
const int INF = 1000000000;

void bellman_ford()
{
    vector<int> d (n + 1, INF); //a5ali el initial distances kolaha = 0
law 3ayza agib ay neg cycle 7atta law msh reachable mn el source
    d[v] = 0;//source node
    vector<int> p (n + 1);
    int x; // element updated in the last iteration
    for (int i=0; i<n; ++i)
    {
        x = -1;
        for (int j=0; j<e.size(); ++j)
            if (d[e[j].a] < INF)
                if (d[e[j].b] > d[e[j].a] + e[j].cost)
                {
                    d[e[j].b] = max (-INF, d[e[j].a] + e[j].cost);
                    p[e[j].b] = e[j].a;
                    x = e[j].b;
                }
    }

    if (x == -1) //no -ve cycles
    {
      cout << "no Negative cycles";
    }
    else
    {
        int y = x;
        for (int i=0; i<n; ++i) //arga3 n marat 3shan ab2a mot2akeda en
el node di fl cycle
            y = p[y];

        vector<int> path;
        for (int cur=y; ; cur=p[cur])
        {
            path.push_back (cur);
            if (cur == y && path.size() > 1)
```

```
                break;
        }
        reverse (path.begin(), path.end());

        cout << "Negative cycle: ";
        for (size_t i=0; i<path.size(); ++i)
            cout << path[i] << ' ';
    }
}
```

Articulation points

(a vertex which, when removed along with associated edges, makes the graph disconnected (or more precisely, increases the number of connected components in the graph)) // !! sa3at el articulation points btb2a visited kaza mara

```
const int N = 1e2 + 5;

int id;
bool vis[N];
vector<int> G[N];
set<int> artPoints;
int tin[N], lowLink[N];

void clean(int n)
{
   id = 0;
   artPoints.clear();
   for(int i = 1; i <= n; i++)
       G[i].clear(), vis[i] = tin[i] = lowLink[i] = 0;
}

void tarjan(int u, int par = -1)
{
   vis[u] = true;
   int children = 0;
   tin[u] = lowLink[u] = ++id;
   for(auto &v: G[u])
   {
       if(v == par)
           continue;
```

```
        if(vis[v])
            lowLink[u] = min(lowLink[u], tin[v]);
        else
        {
            children++;
            tarjan(v, u);
            lowLink[u] = min(lowLink[u], lowLink[v]);
            if(lowLink[v] >= tin[u] && par != -1)
                artPoints.insert(u);
        }
    }
    if(par == -1 && children > 1)
        artPoints.insert(u);
}
```

Bridges

(an edge which, when removed, makes the graph disconnected (or more precisely, increases the number of connected components in the graph))

```
const int N = 1e5 + 5;

bool vis[N];
stack<int> st;
int id, bridges;
vector<int> G[N], G2[N];
int tin[N], low[N], tag[N];

void clean(int n)
{
    id = bridges = 0;
    for(int i = 1; i <= n; i++)
    {
        G[i].clear();
        G2[i].clear();
        vis[i] = false;
        low[i] = tin[i] = tag[i] = 0;
    }
```

```
    while(st.size())
        st.pop();
}

void tarjan(int u, int p = -1)
{
    st.push(u);
    vis[u] = true;
    tin[u] = low[u] = ++id;

    for(auto &v: G[u])
    {
        if(v == p)
            continue;
        if(!vis[v])
        {
            tarjan(v, u);
            low[u] = min(low[u], low[v]);
            if(low[v] > tin[u])
                bridges++;
        }
        else
            low[u] = min(low[u], tin[v]);
    }
    int node = -1;
    if(low[u] == tin[u])
    {
        while(node != u)
        {
            node = st.top(), st.pop();
            tag[node] = u;
        }
    }
}
```

Mo on trees

```
struct query
{
   bool addLCA;
   int l, r, lca, i;
   int64_t ord;

   inline int64_t hilbertOrder(int x, int y, int pow, int rotate)
   {
       if(pow == 0)
       {
           return 0;
       }
       int hpow = 1 << (pow - 1);
       int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 :
2);
       seg = (seg + rotate) & 3;
       const int rotateDelta[4] = {3, 0, 0, 1};
       int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
       int nrot = (rotate + rotateDelta[seg]) & 3;
       int64_t subSquareSize = int64_t(1) << (2 * pow - 2);
       int64_t ans = seg * subSquareSize;
       int64_t add = hilbertOrder(nx, ny, pow - 1, nrot);
       ans += (seg == 1 || seg == 2) ? add : (subSquareSize - add - 1);
       return ans;
   }

   query(bool addLca, int l, int r, int lca, int i) : addLCA(addLca),
l(l), r(r), lca(lca), i(i)
   { ord = hilbertOrder(l, r, 21, 0); }

   inline bool operator<(const query &a) const
   {
       return ord < a.ord;
   }
};


bool vis[N];

void add(int node)
{

}
```

```
void remove(int node)
{

}

void modify(int node)
{
   if(vis[node])
       remove(node);
   else
       add(node);

   vis[node] ^= true;
}

int getAns()
{

}

vector<int> moAlgo(vector<query> &queries)
{
   int curL = 1, curR = 0, l, r;
   vector<int> ret(queries.size());
   sort(queries.begin(), queries.end());

   for(auto &j: queries)
   {
       l = j.l, r = j.r;
       while(curR < r)
           modify(arr[++curR]);
       while(curL > l)
           modify(arr[--curL]);

       while(curR > r)
           modify(arr[curR--]);
       while(curL < l)
           modify(arr[curL++]);

       if(j.addLCA)
           modify(j.lca);

       ret[j.i] = getAns();

       if(j.addLCA)
           modify(j.lca);
```

```
    }
    return ret;
}

void doWork()
{
    int n;
    cin >> n;

    for(int i = 0, u, v, w; i < n - 1; i++)
        cin >> u >> v, G[u].push_back(v), G[v].push_back(u);

    int q;
    cin >> q;
    LCA lca(n);
    vector<query> Q;
    for(int i = 0, u, v; i < q; i++)
    {
        cin >> u >> v;
        if(lca.lvl[u] > lca.lvl[v])
            swap(u, v);

        int LCA = lca.lca(u, v);
        if(LCA == u)
            Q.emplace_back(query(0, lca.tin[u], lca.tin[v], LCA, i));
        else
        {
            int l = lca.tout[u], r = lca.tin[v];
            if(lca.tout[u] > lca.tin[v])
                l = lca.tout[v], r = lca.tin[u];
            Q.emplace_back(query(1, l, r, LCA, i));
        }
    }
```

Euler shit

1- Euler Circuit

Start from node A
Visited every edge exactly once
Return to node A

-> Undirected
1- Connected graph
2- Even degree for each node

-> Directed
1- Connected graph
2- In degree equals out degree for each node

2- Euler Path

Start from node A
Visited every edge exactly once
Return to node B

-> Undireced
1- Connected graph
2- Even degree for each node except 2 nodes are odd

-> Directed
1- Connected graph
2- In degree equals out degree for each node except 2 nodes:
   1) in = out + 1
   2) in = out - 1

Strings

Trie

```
struct Node
{
   int freq = 0;
   Node *ptr[26] = {};
};

struct Trie
{
   Node *root = new Node();

   void insert(string &s)
   {
```

```
        int lvl;
        Node *cur = root;
        for(auto &it: s)
        {
            lvl = it - '0';
            if(!cur->ptr[lvl])
                cur->ptr[lvl] = new Node();
            cur->ptr[lvl]->freq++;
            cur = cur->ptr[lvl];
        }
        // cur -> isLeaf = true
    }
};
```

Trie (Better Time Complexity, Worst Memory)

```
struct trie
{
    int head[maxnode]; // id leftmost child le node i
    int next[maxnode]; // el sibling bt3 node i el 3ala ymino 3ala tool
    char ch[maxnode]; // el char el fl node di
    int leaf[maxnode], freq[maxnode], sz;
    void init() { sz = 1; leaf[0] = head[0] = next[0] = freq[0] = 0; }
    void insert(string& s)
    {
      int u = 0, v;
      freq[0]++;
      for(int i = 0; i < s.size(); i ++)
      {
          bool found = 0;
          for(v = head[u]; v; v = next[v])
          {
              if(ch[v] == s[i])
              {
                  found = 1;
                  break;
              }
          }
          if(!found)
          {
```

```
                v = sz ++;
                freq[v] = leaf[v] = 0;
                ch[v] = s[i];
                next[v] = head[u];
                head[u] = v;
                head[v] = 0;
            }
            u = v;
            freq[u]++;
            if(i == s.size() - 1)
                leaf[u]++;
        }
    }
    void dfs(int node)
    {
        for(int i = head[node]; i; i = next[i])
            dfs(i);
    }
};
```

Manacher (number of palindromes in a string)

```
vector<int> manacherOdd(string &s)
{
   int n = s.size();
   vector<int> d1(n);
   for(int i = 0, l = 0, r = -1; i < n; i++)
   {
       int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
       while(0 <= i - k && i + k < n && s[i - k] == s[i + k])
           k++;
       d1[i] = k--;
       if(i + k > r)
       {
           l = i - k;
           r = i + k;
       }
   }
   return d1;
```

```
}

vector<int> manacherEven(string &s)
{
   int n = s.size();
   vector<int> d2(n);
   for(int i = 0, l = 0, r = -1; i < n; i++)
   {
       int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
       while(0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k])
           k++;
       d2[i] = k--;
       if(i + k > r)
       {
           l = i - k - 1;
           r = i + k;
       }
   }
   return d2;
}
```

KMP

```
vector<int> prefix_function(string &s)
{
   int n = (int) s.length();
   vector<int> pi(n);
   for(int i = 1; i < n; i++)
   {
       int j = pi[i - 1];
       while(j > 0 && s[i] != s[j])
           j = pi[j - 1];
       if(s[i] == s[j])
           j++;
       pi[i] = j;
   }
   return pi;
}
```

Z-Function

```
vector<int> zFunction(string& s)
{
    int n = (int) s.length();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i)
    {
        if (i <= r)
            z[i] = min (r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}
```

Aho-corasick

```
struct AhoCorasick
{
   int states = 0;
   vector<int> pi;
   vector<vector<int>> trie, patterns, dp;
   AhoCorasick(int n, int m = 26)
   {
       pi = vector<int>(n + 5, -1);
       patterns = vector<vector<int>>(n + 1);
       dp = vector<vector<int>>(n + 5, vector<int>(m, -1));
       trie = vector<vector<int>>(n + 5, vector<int>(m,
-1));
   }

   AhoCorasick(vector<string> &p, int n, int m = 26)
   {
       /*
        * MAKE SURE THAT THE STRINGS IN P ARE UNIQUE
        * N is the summation of sizes of p
        * M is the number of used alphabet
       */

       pi = vector<int>(n + 5, -1);
       patterns = vector<vector<int>>(n + 1);
       dp = vector<vector<int>>(n + 5, vector<int>(m, -1));
       trie = vector<vector<int>>(n + 5, vector<int>(m,
-1));

       for(int i = 0; i < p.size(); i++)
           insert(p[i], i);
       build();
   }

   void insert(string &s, int idx)
   {
```

```
        int cur = 0;
        for(auto &it: s)
        {
            if(trie[cur][it - 'a'] == -1)
                trie[cur][it - 'a'] = ++states;
            cur = trie[cur][it - 'a'];
        }
        patterns[cur].push_back(idx);
    }

    void build()
    {
        queue<int> q;
        for(int i = 0; i < 26; i++)
        {
            if(trie[0][i] != -1)
                pi[trie[0][i]] = 0, q.push(trie[0][i]);
            else
                trie[0][i] = 0;
        }

        while(q.size())
        {
            int cur = q.front();
            q.pop();
            for(int i = 0; i < 26; i++)
            {
                if(trie[cur][i] == -1)
                    continue;

                int f = pi[cur];
                while(trie[f][i] == -1)
                    f = pi[f];

                f = trie[f][i];
                pi[trie[cur][i]] = f;

patterns[trie[cur][i]].insert(patterns[trie[cur][i]].end(),
```

```
patterns[f].begin(), patterns[f].end());
                q.push(trie[cur][i]);
            }
        }
    }

    int nextState(int trieNode, char nxt)
    {
        int &ret = dp[trieNode][nxt - 'a'];
        if(~ret)
            return ret;
        int cur = trieNode;
        if(trie[cur][nxt - 'a'] == -1)
            return ret = nextState(pi[cur], nxt);

        return ret = trie[cur][nxt - 'a'];
    }

    vector<int> search(string &s, int n)
    {
        int cur = 0;
        vector<int> ret(n);
        for(int i = 0; i < s.length(); i++)
        {
            cur = nextState(cur, s[i]);
            if(cur == 0 || patterns[cur].empty())
                continue;

            // patterns vector have every pattern that is
matched in this node
            // matched: the last index in the pattern is
index i
            for(auto &it: patterns[cur])
                ret[it]++;
        }
        return ret;
    }
};
```

Suffix Automaton

```
struct suffixAutomaton
{

    int last = 0, cntState = 1;
    struct state
    {
        int len = 0, link = -1, cnt = 0;
        map<char, int> next;
    };

    state st[N * 2];
    ll dp[N * 2];

    void addChar(char c)
    {

        int p = last;
        int cur = cntState++;
        st[cur].len = st[last].len + 1;
        st[cur].cnt = 1;

        while(p != -1 && st[p].next.count(c) == 0)
        {
            st[p].next[c] = cur;
            p = st[p].link;
        }

        if(p == -1)
        {
            st[cur].link = 0;
        }
        else
        {
            int q = st[p].next[c];
            if(st[q].len == st[p].len + 1)
            {
```

```
            st[cur].link = q;
        }
        else
        {
            int clone = cntState++;
            st[clone].link = st[q].link;
            st[clone].len = st[p].len + 1;
            st[q].link = st[cur].link = clone;
            st[clone].next = st[q].next;
            st[clone].cnt = 0;
            while(p != -1 && st[p].next[c] == q)
            {
                st[p].next[c] = clone;
                p = st[p].link;
            }
        }
    }
    last = cur;
}

bool checkForOccurrence(string str)
{

    int cur = 0;
    for(int i = 0; i < str.size(); i++)
    {
        if(st[cur].next.count(str[i]))
        {
            cur = st[cur].next[str[i]];
        }
        else
        {
            return false;
        }
    }

    return true;
}
```

```
    ll numberOfSubstrings()
    {

        ll ans = 0;

        for(int i = 1; i < cntState; i++)
        {
            ans += st[i].len - st[st[i].link].len;
        }

        return ans;

    }

    ll lengthOfSubstrings()
    {

        auto getSum = [&](ll x) -> ll {
            return x * (x + 1) / 2;
        };

        ll ans = 0;

        for(int i = 1; i < cntState; i++)
        {
            ans += getSum(st[i].len) -
getSum(st[st[i].link].len);
        }

        return ans;

    }

    void numberOfOccPreProcess()
    {

        static bool visisted = false;
```

```
        if(visisted)return;
        visisted = true;

        vector<pair<int, int>> v;

        for(int i = 0; i < cntState; i++)
            v.push_back(pair<int, int>(st[i].len, i));

        sort(v.rbegin(), v.rend());

        for(int i = 0; i < cntState; i++)
        {
            if(st[v[i].second].link > 0)
                st[st[v[i].second].link].cnt +=
st[v[i].second].cnt;
            dp[v[i].second] = st[v[i].second].cnt;
            for(auto x: st[v[i].second].next)
            {
                dp[v[i].second] += dp[x.second];
            }
        }

    }

    ll numberOfOcc(string s)
    {

        numberOfOccPreProcess();

        int cur = 0;
        for(char c: s)
        {
            if(st[cur].next.count(c))
            {
                cur = st[cur].next[c];
            }
            else
                return 0;
```

```
        }
        return st[cur].cnt;
    }

    void dfs(int node, ll &k, string &ans)
    {
        k -= st[node].cnt;
        if(k <= 0)return;
        for(auto x: st[node].next)
        {
            if(dp[x.second] < k)
            {
                k -= dp[x.second];
            }
            else
            {
                ans.push_back(x.F);
                dfs(x.second, k, ans);
                return;
            }

        }

    }

    string getKthLex(ll k)
    {
        numberOfOccPreProcess();
        if(dp[0] < k)
            return "No such line.";
        string ans = "";
        dfs(0, k, ans);
        return ans;
    }

};
```

KMP Palindrome

```
while (cin >> s)
{
    int s3[10001];
    s2 = s;
    s3[0] = 0;
    reverse(s2.begin(), s2.end());
    for(int i = 1, j = 0; i < sz(s); i++)
    {
        while(j > 0 && s2[j] != s2[i])
            j = s3[j - 1];
        if(s2[j] == s2[i])
            j++;
        s3[i] = j;
    }
    int a = 0;
    for(int i = 0; i < sz(s); i++)
    {
        while(a> 0 && s[i] != s2[a])
            a = s3[a - 1];
        if(s[i] == s2[a])
            a++ ;
        if(a == sz(s))
            a = s3[a - 1];
    }
}
```

Hashing

```
const int N = 1e6 + 5, P1 = 29, P2 = 31, MOD1 = 1e9 + 7, MOD2 = 1e9 + 9;
int pw1[N], inv1[N], pw2[N], inv2[N];

int add(int a, int b, int mod)
{
   return (a + b) % mod;
}

int sub(int a, int b, int mod)
{
   return ((a - b) % mod + mod) % mod;
}

int mul(int a, int b, int mod)
{
   return 1ll * a * b % mod;
}

int fp(int base, int power, int mod)
{
   if(!power)
       return 1;

   int ret = fp(base, power >> 1, mod);
   ret = mul(ret, ret, mod);

   if(power & 1)
       ret = mul(ret, base, mod);

   return ret;
}

struct Hash
{
   vector<pair<int, int>> prefix;

   Hash(const string &s)
   {
       prefix.resize(s.size() + 1);
       for(int i = 0; i < int(s.size()); ++i)
           prefix[i + 1] = make_pair(add(prefix[i].first, mul(pw1[i],
s[i] - 'a' + 1, MOD1), MOD1),
                                     add(prefix[i].second, mul(pw2[i],
```

```
s[i] - 'a' + 1, MOD2), MOD2));
    }

    int size() const
    {
        return prefix.size() - 1;
    }

    pair<int, int> getHash() const
    {
        return prefix.back();
    }

    pair<int, int> getRange(int l, int r) const
    {
        return make_pair(mul(inv1[l], sub(prefix[r + 1].first,
prefix[l].first, MOD1), MOD1),
                         mul(inv2[l], sub(prefix[r + 1].second,
prefix[l].second, MOD2), MOD2));
    }

    static void prepare()
    {
        pw1[0] = inv1[0] = pw2[0] = inv2[0] = 1;
        int iv1 = fp(P1, MOD1 - 2, MOD1);
        int iv2 = fp(P2, MOD2 - 2, MOD2);
        for(int i = 1; i < N; ++i)
        {
            pw1[i] = mul(pw1[i - 1], P1, MOD1);
            inv1[i] = mul(inv1[i - 1], iv1, MOD1);
            pw2[i] = mul(pw2[i - 1], P2, MOD2);
            inv2[i] = mul(inv2[i - 1], iv2, MOD2);
        }
    }
};
Int main()
{
Hash::prepare();
}
```

Suffix Array

```
struct SuffixArray
{
   string S;
   vector<int> sa, lcp;
   SuffixArray(string& s, int lim = 256)
   {
       S = s;
       int n = s.size() + 1, k = 0, a, b;
       vector<int> x(s.begin(), s.end() + 1), y(n), ws(max(n, lim)),
rank(n);
       sa = lcp = y, iota(sa.begin(), sa.end(), 0);
       for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p)
       {
           p = j, iota(y.begin(), y.end(), n - j);
           for(int i = 0; i < n; i++)
           {
               if (sa[i] >= j)
                   y[p++] = sa[i] - j;
           }

           fill(ws.begin(), ws.end(), 0);

           for(int i = 0; i < n; i++) ws[x[i]]++;
           for(int i = 1; i < lim; i++) ws[i] += ws[i - 1];
           for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];

           swap(x, y), p = 1, x[sa[0]] = 0;
           for(int i = 1; i < n; i++)
               a = sa[i - 1], b = sa[i], x[b] = (y[a] == y[b] && y[a +
j] == y[b + j]) ? p - 1 : p++;
       }

       for(int i = 1; i < n; i++) rank[sa[i]] = i;
       for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
           for (k && k--, j = sa[rank[i] - 1];
                   s[i + k] == s[j + k]; k++);
   }
};
```

$$\sum_{i=0}^{n-1} (m - p[i]) - \sum_{i=0}^{n-2} lcp[i] = \frac{n^2 + n}{2} - \sum_{i=0}^{n-2} lcp[i]$$

Z-Algorithm

```
int Z[5000005];

void Z_algo(string t, string p) // text, pattern
{
    string s = p + '$' + t;
    int l = 0, r = 0;
    for(int i = 1; i < s.size(); i ++)
    {
        if(i > r) //outside of the box
        {
            l = r = i;
            while(r < s.size() && s[r] == s[r - l])
                r++;
            r--;
            Z[i] = r - l + 1;
        }
        else  // inside the box
        {
            int u = i - l;
            if(Z[u] + i - 1 < r) //value does not stretch till right
bound then copy
                Z[i] = Z[u];
            else // check if we can match more
            {
                l = i;
                while(r < s.size() && s[r] == s[r - l])
                    r++;
                r--;
                Z[i] = r - l + 1;
            }
        }
    }
}
```

KMP Pre (automaton)

```
vector<vector<int>> aut;

void compute_automaton(string s)
{
   s += '#';
   int n = s.size();
   vector<int> pi = prefix_function(s);
   aut.assign(n, vector<int>(26));
   for(int i = 0; i < n; i++)
   {
       for(int c = 0; c < 26; c++)
       {
           if(i > 0 && 'a' + c != s[i])
               aut[i][c] = aut[pi[i - 1]][c];
           else
               aut[i][c] = i + ('a' + c == s[i]);
       }
   }
}
// compute the automaton for pattern T to determine the fail position of
KMP in O(1) complexity
// aut[index][letter] = the fail position of KMP aka the next character
to match
```

DFA

```
int dfa[26][5000005], last;
string p, t;

void DFA()
{
    dfa[p[0] - 'a'][0] = 1; // law shoft awl 7arf wana f state 0 haroo7
l state 1
    int x = 0;
    for(int j = 1; j < p.size(); j ++)
    {
        for(int i = 0; i < 26; i ++)
            dfa[i][j] = dfa[i][x];  // ba copy kol el f state X 3andi -
el mismatch
```

```
        dfa[p[j] - 'a'][j] = j + 1; // match case fa batn2el 3al state
el b3di
        if(dfa[p[j] - 'a'][j] == p.size())
            last = dfa[p[j] - 'a'][x]; // last da el el mafrood aro7lo
law wslt l a5er el string 3shan law fi hetta already matched akamel
3aleha( law fi overlapping "foobarfoo" : foo )
        x = dfa[p[j] - 'a'][x]; // update restart state
    }
}
void Search_Dfa()
{
    int j = 0;
    for(int i = 0; i < t.size(); i ++)
    {
        j = dfa[t[i] - 'a'][j];
        if(j == p.size())
            cout << i - j + 1 << endl,
        j = last;
    }
}
```

Count Unique Substrings

```
int countUniqueSubs(string& s)
{
   ll ans = 0;
   int n = s.length();
   string t = "", rev;
   for(int i = 0; i < n; i++)
   {
       t += s[i], rev = t, reverse(rev.begin(), rev.end());
       auto z = zFunction(rev);
       ans += (t.size() - *max_element(z.begin(), z.end()));
   }
   return ans;
}
```

DP

LIS

```
int lis(vector<int>& a)
{
    vector<int>ans;
    int n=a.size();
    for (int i=0;i<n;i++)
    {
        if (ans.size()==0||ans.back()<=a[i])
        {
            ans.push_back(a[i]);
            continue;
        }
        int idx=upper_bound(ans.begin(),ans.end(),a[i])-ans.begin();
        ans[idx]=a[i];
    }
    return ans.size();
}
```

Matrix Power

$F_N = T^{N/2}.F_1$, if N is odd, $F_N = T_{\text{odd}}.T^{(N-1)/2}.F_1$, otherwise.

```
#define matrix vector<vector<int>>
const ll mod2 = 1ll * mod * mod;

int add(int a, int b){
      return (a + b) % mod;
}
 int mul(int a, int b){
      return 1ll * a * b % mod;
}

matrix mul(matrix &a, matrix &b){
      int n = a.size(), k = a[0].size();
      int k2 = b.size(), m = b[0].size();
      assert(k == k2);
      matrix ret(n, vector<int> (m, 0));
      for(int i = 0; i < n; i++){
      for(int j = 0; j < m; j++){
            ll tmp = 0;
            for(int k = 0; k < k2; k++){
```

```
                    tmp += 1ll * a[i][k] * b[k][j];
                    while(tmp >= mod2) tmp -= mod2;
                }
                ret[i][j] = tmp % mod;
        }
        }
        return ret;
}

matrix I(int n){
        matrix ret(n, vector<int> (n, 0));
        for(int i = 0; i < n; i++) ret[i][i] = 1;
        return ret;
}

matrix fp(matrix &b, ll p){
        if(p == 0) return I(b.size());
        auto ret = fp(b, p >> 1);
        ret = mul(ret, ret);
        if(p & 1) ret = mul(ret, b);
        return ret;
}
```

CHT

```
struct Line
{
   mutable ll k, m, p;

   bool operator<(const Line &o) const
   { return k < o.k; }

   bool operator<(ll x) const
   { return p < x; }
};

struct CHT : multiset<Line, less<>>
{
   // (for doubles, use inf = 1/.0, div(a,b) = a/b)
   static const ll inf = LLONG_MAX;

   ll div(ll a, ll b)
```

```
    { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }

    bool isect(iterator x, iterator y)
    {
        if(y == end()) return x->p = inf, 0;
        if(x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }

    void add(ll k, ll m)
    {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while(isect(y, z)) z = erase(z);
        if(x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }

    ll query(ll x)
    {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

Data Structures

Segment Tree

```
struct Node {

};

struct segTree {
   Node tree[N * 4];
   Node neutral = Node();

   Node merge(Node u, Node v) {

   }

   Node single(int x) {

   }

   void build(int x, int l, int r, vector<int> &v) {
       if (l == r) {
           tree[x] = single(v[l]);

           return;
       }

       int m = (l + r) >> 1;

       build(x * 2, l, m, v);
       build(x * 2 + 1, m + 1, r, v);

       tree[x] = merge(tree[x * 2], tree[x * 2 + 1]);
   }

   void set(int x, int l, int r, int i, int v) {
       if (l == r) {
           tree[x] = single(v);

           return;
       }
```

```
        int m = (l + r) >> 1;

        if (i <= m)
            set(x * 2, l, m, i, v);
        else
            set(x * 2 + 1, m + 1, r, i, v);

        tree[x] = merge(tree[x * 2], tree[x * 2 + 1]);
    }

    Node query(int x, int lX, int rX, int l, int r) {
        if (lX > r || rX < l)
            return neutral;

        if (lX >= l && rX <= r)
            return tree[x];

        int m = (lX + rX) >> 1;

        Node u = query(x * 2, lX, m, l, r);
        Node v = query(x * 2 + 1, m + 1, rX, l, r);

        return merge(u, v);
    }
} st;
```

Segment Tree Lazy

```
struct Node {
    int sum = 0;
};

struct segTree {
    int lazy[N * 4];
    Node tree[N * 4];
    Node neutral = Node();

    Node merge(Node u, Node v) {
        return {u.sum + v.sum};
    }

    Node single(int x) {
        return {x};
    }

    void propagate(int x, int lX, int rX) {
```

```
        tree[x].sum += (rX - lX + 1) * lazy[x];

        if (lX != rX) {
            lazy[x * 2] += lazy[x];
            lazy[x * 2 + 1] += lazy[x];
        }

        lazy[x] = 0;
    }

    void build(int x, int l, int r, vector<int> &v) {
        if (l == r) {
            tree[x] = single(v[l]);

            return;
        }

        int m = (l + r) >> 1;

        build(x * 2, l, m, v);
        build(x * 2 + 1, m + 1, r, v);

        tree[x] = merge(tree[x * 2], tree[x * 2 + 1]);
    }

    void update(int x, int lX, int rX, int l, int r, int val) {
        propagate(x, lX, rX);

        if (lX > r || rX < l)
            return;

        if (lX >= l && rX <= r) {
            tree[x].sum += (rX - lX + 1) * val;

            if (lX != rX) {
                lazy[x * 2] += val;
                lazy[x * 2 + 1] += val;
            }

            return;
        }

        int m = (lX + rX) >> 1;

        update(x * 2, lX, m, l, r, val);
        update(x * 2 + 1, m + 1, rX, l, r, val);
```

```
        tree[x] = merge(tree[x * 2], tree[x * 2 + 1]);
    }

    Node query(int x, int lX, int rX, int l, int r) {
        if (lX > r || rX < l)
            return neutral;

        propagate(x, lX, rX);

        if (lX >= l && rX <= r)
            return tree[x];

        int m = (lX + rX) >> 1;

        Node u = query(x * 2, lX, m, l, r);
        Node v = query(x * 2 + 1, m + 1, rX, l, r);

        return merge(u, v);
    }
} st;
```

Persistent Segment Tree

```
struct Node {
    int sum;
    Node *l, *r;

    Node(int val) : l(nullptr), r(nullptr), sum(val) {}

    Node(Node *l, Node *r) : l(l), r(r), sum(0) {
        if (l) sum += l->sum;
        if (r) sum += r->sum;
    }
};

Node* build(int a[], int tl, int tr) {
    if (tl == tr)
        return new Node(a[tl]);

    int tm = (tl + tr) / 2;

    return new Node(build(a, tl, tm), build(a, tm + 1, tr));
}

int getSum(Node* v, int tl, int tr, int l, int r) {
    if (l > r)
```

```
        return 0;

    if (l == tl && tr == r)
        return v->sum;

    int tm = (tl + tr) / 2;

    return getSum(v->l, tl, tm, l, min(r, tm)) + getSum(v->r, tm + 1, tr,
max(l, tm + 1), r);
}

Node* update(Node* v, int tl, int tr, int pos, int new_val) {
    if (tl == tr)
        return new Node(new_val);

    int tm = (tl + tr) / 2;

    if (pos <= tm)
        return new Node(update(v->l, tl, tm, pos, new_val), v->r);
    else
        return new Node(v->l, update(v->r, tm + 1, tr, pos, new_val));
}
```

Monotonic Queue

```
struct monotonicQ
{
    stack<pair<int, int>> s1, s2;
    int findMin()
    {
        int ans;
        if (s1.empty() || s2.empty())
            ans = s1.empty()? s2.top().second: s1.top().second;
        else
            ans = min(s1.top().second, s2.top().second);
        return ans;
    }
    void add(int x)
    {
        int minimum = s1.empty()? x: min(x, s1.top().second);
        s1.push({x, minimum});
    }
    int remove()
    {
        if (s2.empty())
        {
```

```
            while (!s1.empty())
            {
                int element = s1.top().first;
                s1.pop();
                int minimum = s2.empty()? element: min(element,
s2.top().second);
                s2.push({element, minimum});
            }
        }
        int ans = s2.top().first;
        s2.pop();
        return ans;
    }
};
```

Fenwick Tree

```
struct FenwickTree {
    vector<int> a, b;

    FenwickTree(int n) {
        a.assign(n, 0);
        b.assign(n, 0);
    }

    FenwickTree(vector<int> &v) {
        a.assign(v.size(), 0);
        b.assign(v.size(), 0);

        for (int i = 0; i < v.size(); i++)
            update(i, i, v[i]);
    }

    int sum(vector<int> &v, int r) {
        int ret = 0;

        for (; r >= 0; r = (r & (r + 1)) - 1)
            ret += v[r];

        return ret;
    }

    int solve(int i) {
        return sum(a, i) * i - sum(b, i);
    }

    int query(int l, int r) {
```

```
        return solve(r) - solve(l - 1);
    }

    void set(vector<int> &v, int i, int d) {
        for (; i < v.size(); i = i | (i + 1))
            v[i] += d;
    }

    void update(int l, int r, int d) {
        set(a, l, d);
        set(a, r + 1, -d);

        set(b, l, d * (l - 1));
        set(b, r + 1, -d * r);
    }
};
```

Sparse Table

```
struct SparseTable {
    vector<int> pre;
    vector<vector<int>> g;

    SparseTable(vector<int> &v) {
        g.resize(N);
        pre.resize(N);
        for (int i = 0; i < N; i++)
            g[i].resize(LOG);

        int n = v.size();

        pre[1] = 0;
        for (int i = 2; i <= n; i++)
            pre[i] = pre[i >> 1] + 1;

        for (int i = 0; i < n; i++)
            g[i][0] = v[i];

        for (int k = 1; k < LOG; k++) {
            for (int i = 0; i + (1 << k) - 1 < n; i++)
                g[i][k] = min(g[i][k - 1], g[i + (1 << (k - 1))][k - 1]);
        }
    }

    int query(int l, int r) {
        int d = r - l + 1, k = pre[d];
        return min(g[l][k], g[r - (1 << k) + 1][k]);
```

```
    }
};
```

2D Sparse Table

```
int g[N][N];

struct SparseTable2D {
    int lg2[N];
    int st[N][N][LG][LG];

    void pre(int n, int m) {
        for (int i = 2; i < N; i++)
            lg2[i] = lg2[i >> 1] + 1;

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                st[i][j][0][0] = g[i][j];
            }
        }
    }

    void build(int n, int m) {
        pre(n, m);

        for (int a = 0; a < LG; a++) {
            for (int b = 0; b < LG; b++) {
                if (a + b == 0)
                    continue;

                for (int i = 0; i + (1 << a) <= n; i++) {
                    for (int j = 0; j + (1 << b) <= m; j++) {
                        if (!a)
                            st[i][j][a][b] = max(st[i][j][a][b - 1],
st[i][j + (1 << (b - 1))][a][b - 1]);
                        else
                            st[i][j][a][b] = max(st[i][j][a - 1][b], st[i
+ (1 << (a - 1))][j][a - 1][b]);
                    }
                }
            }
        }
    }
    int query(int x1, int y1, int x2, int y2) {
        x2++, y2++;

        int a = lg2[x2 - x1], b = lg2[y2 - y1];
```

```
        return max(
                max(st[x1][y1][a][b], st[x2 - (1 << a)][y1][a][b]),
                max(st[x1][y2 - (1 << b)][a][b], st[x2 - (1 << a)][y2 -
(1 << b)][a][b])
        );
    }
};
```

DSU

```
struct DSU {
    vector<int> sz, par;

    DSU() {
        sz.resize(N);
        par.resize(N);

        for (int i = 0; i < N; i++)
            par[i] = i, sz[i] = 1;
    }

    int findSet(int u) {
        if (u == par[u])
            return u;

        return par[u] = findSet(par[u]);
    }

    void unionSet(int u, int v) {
        u = findSet(u);
        v = findSet(v);

        if (u != v) {
            if (sz[u] < sz[v])
                swap(u, v);

            par[v] = u;
            sz[u] += sz[v];
        }
    }
};
```

DSU Bipartite

```
struct DSU {
    vector<int> sz;
    vector<int> bipartite;
    vector<pair<int, int>> par;

    DSU() {
        sz.resize(N);
        par.resize(N);
        bipartite.resize(N);

        for (int i = 0; i < N; i++)
            par[i] = {i, 0}, sz[i] = 1;
    }

    pair<int, int> findSet(int u) {
        if (u != par[u].first) {
            int parity = par[u].second;

            par[u] = findSet(par[u].first);

            par[u].second ^= parity;
        }

        return par[u];
    }

    void unionSet(int u, int v) {
        pair<int, int> pA = findSet(u);

        u = pA.first;

        int x = pA.second;

        pair<int, int> pB = findSet(v);

        v = pB.first;

        int y = pB.second;

        if (u == v) {
            if (x == y)
                bipartite[u] = 0;
        }
        else {
            if (sz[u] < sz[v])
                swap(u, v);
```

```
            par[v] = make_pair(u, x ^ y ^ 1);

            bipartite[u] &= bipartite[v];

            sz[u] += sz[v];
        }
    }

    bool isBipartite(int u) {
        return bipartite[findSet(u).first];
    }
};
```

DSU Rollback

```
struct Query {
    int t, u, v;
};

struct Elem {
    int u, v, szU, cnt;
};

struct DSURollback {
    int cnt;
    stack<Elem> st;
    vector<int> sz, par;
    map<int, vector<pair<int, int>>> g;

    DSURollback(int n) {
        cnt = n;

        par.resize(n + 1);

        sz.resize(n + 1, 1);

        iota(par.begin(), par.end(), 0);
    }

    void rollback(int x) {
        while (st.size() > x) {
            auto e = st.top();

            st.pop();

            cnt = e.cnt;
```

```
        par[e.v] = e.v;

        sz[e.u] = e.szU;
    }
}

int findSet(int u) {
    return par[u] == u ? par[u] : findSet(par[u]);
}

void update(int u, int v) {
    st.push({u, v, sz[u], cnt});

    cnt--;

    par[v] = u;

    sz[u] += sz[v];
}

void unionSet(int u, int v) {
    u = findSet(u);

    v = findSet(v);

    if (u != v) {
        if (sz[u] < sz[v])
            swap(u, v);

        update(u, v);
    }
}

void solve(int x, int l, int r) {
    int cur = st.size();

    for (auto i: g[x])
        unionSet(i.first, i.second);

    if (l == r) {
        cout << cnt << ' ';

        rollback(cur);

        return;
    }
```

```
        int m = (l + r) >> 1;

        solve(x * 2, l, m);

        solve(x * 2 + 1, m + 1, r);

        rollback(cur);
    }

    void traverse(int x, int lX, int rX, int l, int r, int u, int v) {
        if (rX < l || lX > r)
            return;

        if (lX >= l && rX <= r) {
            g[x].emplace_back(u, v);

            return;
        }

        int m = (lX + rX) >> 1;

        traverse(x * 2, lX, m, l, r, u, v);

        traverse(x * 2 + 1, m + 1, rX, l, r, u, v);
    }

    void build(vector<Query> &queries) {
        int q = queries.size();

        map<pair<int, int>, vector<pair<int, int>>> mp;

        for (int i = 0; i < queries.size(); i++) {
            auto cur = queries[i];

            if (cur.u > cur.v)
                swap(cur.u, cur.v);

            if (cur.t == 1)
                mp[{cur.u, cur.v}].emplace_back(i, queries.size());
            else {
                mp[{cur.u, cur.v}].back().second = i - 1;

                traverse(1, 0, q - 1, mp[{cur.u, cur.v}].back().first,
mp[{cur.u, cur.v}].back().second, cur.u, cur.v);
            }
        }
```

```
        for (auto i: mp) {
            for (auto j: i.second) {
                if (j.second == q)
                    traverse(1, 0, q - 1, j.first, q - 1, i.first.first,
i.first.second);
            }
        }
    }
};
```

MO

```
struct query
{
    int64_t ord;
    int l, r, i;

    query(int l1, int r1, int i1)
    {
        l = l1, r = r1, i = i1;
        ord = hilbertOrder(l, r, 21, 0);
    }

    inline int64_t hilbertOrder(int x, int y, int pow, int rotate)
    {
        if(pow == 0)
        {
            return 0;
        }
        int hpow = 1 << (pow - 1);
        int seg = (x < hpow) ? ((y < hpow) ? 0 : 3) : ((y < hpow) ? 1 :
2);
        seg = (seg + rotate) & 3;
        const int rotateDelta[4] = {3, 0, 0, 1};
        int nx = x & (x ^ hpow), ny = y & (y ^ hpow);
        int nrot = (rotate + rotateDelta[seg]) & 3;
        int64_t subSquareSize = int64_t(1) << (2 * pow - 2);
        int64_t ans = seg * subSquareSize;
        int64_t add = hilbertOrder(nx, ny, pow - 1, nrot);
        ans += (seg == 1 || seg == 2) ? add : (subSquareSize - add - 1);
        return ans;
    }

    bool operator<(const query &q)
```

```
    {
        return ord < q.ord;
    }


//    bool operator<(const query &q)
//    {
//        if(q.l / SQ != l / SQ)
//            return l / SQ < q.l / SQ;
//
//        return r < q.r;
//    }


};

void add(int i)
{

}

void remove(int i)
{

}

ll getAns()
{

}

vector<ll> moAlgo(vector<query> &queries)
{
    int curL = 1, curR = 0, l, r;
    vector<ll> ret(queries.size() + 1);
    sort(queries.begin(), queries.end());

    for(auto &j: queries)
    {
        l = j.l, r = j.r;
        while(curR < r)
            add(++curR);
        while(curL > l)
            add(--curL);

        while(curR > r)
            remove(curR--);
```

```
        while(curL < l)
            remove(curL++);

        ret[j.i] = getAns();
    }
    return ret;
} a, b;
        cin >> a >> b;
        //ans = xor grundy of all the starting positions(all the piles)
        ans ^= solve(a, b);
    }
    cout << (ans ? "Y" : "N");
    return 0;
}
```

MEX

```
bool used[10001];

void MEX()
{
    int mex=0;
    for (int k = 0; k <= (int)se.size() + 1; k++)//se : vector
        used[k] = 0;
    for (int k = 0; k < (int)se.size(); k++)
    {
        if (se[k]<=(int)se.size())
            used[se[k]] =1;
    }
    while (used[mex])
        mex++;
}
```

Note

if we can take 1, 2 or 3 coins from the pile :

grundy(10) = mex({grundy(9), grundy(8),grundy(7)}) //mex of the grundy numbers of the states that i can go to from here

Grundy of losing states = 0, winning states = inf (base case)

Result = xor(grundy of all the starting states/ piles)

Game of Nim

If we have k piles then sum all bits and if all of them is divisible by (k+1) first player lose .

Losing positing xor=0.

If losing is last to move if all numbers is <=1 xor=0 first player wins else normal wins

Misere nim (last one play wins): if all 1 if n even First wins else second
Else normal nim

Articles

Aho corasick applications

1- find all occurrences of a set of strings in a text
2- Finding the lexicographical smallest string of a given length that doesn't match any given strings

A set of strings and a length L is given. We have to find a string of length L, which does not contain any of the string, and derive the lexicographical smallest of such strings.

We can construct the automaton for the set of strings. Let's remember, that the vertices from which we can reach a leaf vertex are the states, at which we have a match with a string from the set. Since in this task we have to avoid matches, we are not allowed to enter such states. On the other hand we can enter all other vertices. Thus we delete all "bad" vertices from the machine, and in the remaining graph of the automaton we find the lexicographical smallest path of length L. This task can be solved in O(L) for example by depth first search.

3- Finding the shortest string containing all given strings

Here we use the same ideas. For each vertex we store a mask that denotes the strings which match at this state. Then the problem can be reformulated as follows: initially being in the state (v=root, mask=0), we want to reach the state (v, mask=2n−1), where n is the number of strings in the set. When we transition from one state to another using a letter, we update the mask accordingly. By running a breath first search we can find a path to the state (v, mask=2n−1) with the smallest length.

4- Finding the lexicographical smallest string of length L containing k strings

As in the previous problem, we calculate for each vertex the number of matches that correspond to it (that is the number of marked vertices reachable using suffix links).

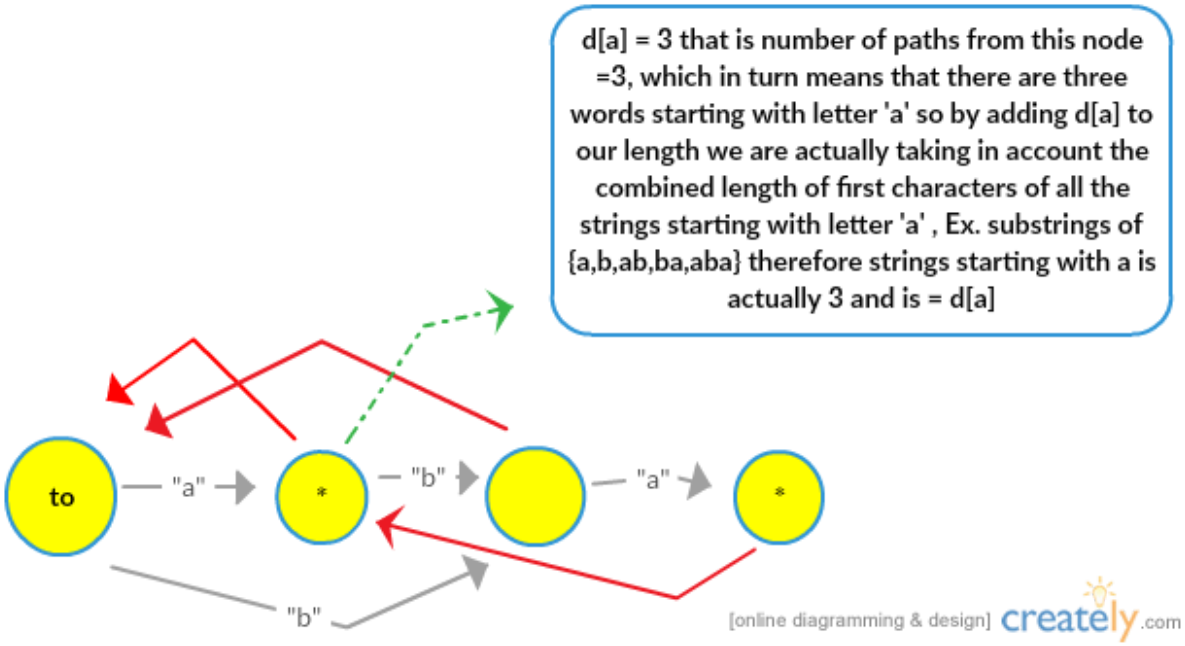
We reformulate the problem: the current state is determined by a triple of numbers (v, len, cnt), and we want to reach from the state (root, 0, 0) the state (v, L, k), where v can be any vertex. Thus we can find such a path using depth first search (and if the search looks at the edges in their natural order, then the found path will automatically be the lexicographical smallest).

Suffix automaton applications

1. Finding number of distinct substrings (implemented above)

2. The Lexicographically kth substring (implemented above)

3.Total Length of all distinct substrings :
For this we need to look at things more closely. Consider




```
vector<int>ans(MAXLEN,0);
int lesub( int ver)
{
    int tp = 0;//To Find Words just add length of words instead of 1
    if(ans[ver])
        return ans[ver];
    for(int i = 0; i < 26; i++)
        if(st[ver].next[i])
            tp = lesub(st[ver].next[i]) + d[st[ver].next[i]];

    ans[ver] = tp;
    return ans[ver];
}
```

4. Smallest Cyclic Shift to obtain lexicographical smallest of All possible

Consider string “aba” then the minimum cyclic shift is ‘1’ because by shifting the pattern by one we get string “aab” which is lexicographically smallest of all possible (“aba” , “aab”, “baa”) . For solving this problem we build the suffix automaton of *“String + String”* Now the problem is reduced to above problem where k = 1 , solving until we get substring of length = length of given string,Don’t worry about the fpos part that is done to find the start position of minimum lexicographical string so that we can find number of shifts.

```
int tp = 0 ;
string s;
int t = s.length();
s = s + s;
void minshift(int ver)
{
    for(int i = 0; i < 26; i++)
        if(st[ver].next[i])
        {
            tp++;
            if(tp == t)
            {
                cout<<st[ver].fpos - t + 2<<endl;
                Break;
            }
            minshift(st[ver].next[i]);
            break;
        }
}
```

5. POSITION OF FIRST OCCURRENCE

Given a text T and we receive request of the form : given a pattern P we need to know the position of first occurrence of P in T .

To solve this we will build the suffix automaton of the Text T , we also need to add in the preprocessing finding fpos (first position of occurrence) for all states of automaton, such that fpos[cur] = len[cur] – 1 and fpos[clone] = fpos[q] , then answer to our query is simply equal to fpos[t] – p.length() + 1 where "t" is the state corresponding to pattern , p is the pattern. Here is the code

```
st[cur].fpos = st[cur].len - 1;
st[clone].fpos = st[q].fpos;
at appropriate positions in sa_extend function */

int firstpostn()
{
    int s_t = 0 ;
    int vtx = 0;
    int u = s_t;
    while(s_t < p.length())
    {
        vtx = st[vtx].next[p[s_t] - 'a'];
        s_t++;
    }
    Cout << st[vtx].fpos - p.length() + 1 << endl;
}
```

6. COUNT NUMBER OF OCCURRENCES

To solve this we need to build suffix automaton of S , along with this we also need to make a preprocessing for each state *v* to find *count[v]* which is equal to the size of endpos(v) since it would not be possible to store all suffixes in endpos(v) , we will instead store its size. For each state , other than initial state or states obtained by cloning we initially assign *count[v]* = 1 . Then we will go over all the states in descending order of their length *len* and calculate *count[v] using count[link[v]]* += *count[v]* . After this answer to query is *count[t]* where *" t is the state of the pattern ". Here is the code which find occurrences of pattern string "pa" in main string "s".*

```
set<pair<int, int> base;
int s_t = 0
/* ADD these 4 lines at appropriate places in sa_extend function
   cnt[cur] = 1;
   base.insert(make_pair(st[cur].len, cur));
   cnt[clone] = 0;
   base.insert(make_pair(st[clone].len,clone));
*/
int repstr()
{
    set::reverse_iterator it;
    for(it = base.rbegin(); it != base.rend(); it++)
        cnt[st[it->second].link] += cnt[it->second];
    int vtx = 0;
    while(s_t<=pa.length()-1)
    {
        vtx = st[vtx].next[pa[s_t] - 'a'];
        S_t++;
    }
    cout << cnt[vtx] << endl;
}
```

7. LONGEST COMMON SUBSTRING OF TWO STRINGS

We now go on line T and look for each prefix of the longest suffix prefix S occurring. In other words, for each position in the line T we want to find of the longest common substring S and T ending at that position. To do this, we will support the two variables: the current state v and the current l length. These two variables will describe the current matching part: its length and state that corresponds to it.

Initially, $p = t_0$ and $l = 0$ i.e empty match .

Now let us consider the character $T[i]$ and we want to recalculate the answer for it.

- If the state of v the automaton has a transition with label $T[i]$, we simply commit this change and increase l by one.
- And if the state v is not of the desired transition, then we should try to shorten the current matching portion for which it is necessary to go to suffix link:
  $v = link(v)$.
  $l = len(v)$.
  Unless we find a new state with required transition or we reach a fictitious

state -1 , we have to go on the suffix link. The answer to the problem will be a maximum of the values l for all time rounds.

```
string lcs (string s, string t)
{
    sa_init();
    for (int i=0; i<(int)s.length(); ++i)
        sa_extend (s[i]);
    int v = 0, l = 0, best = 0, bestpos = 0;
    for (int i = 0; i < (int)t.length(); ++i)
    {
        while (v && ! st[v].next.count(t[i]))
        {
            v = st[v].link;
            l = st[v].length;
        }
        if (st[v].next.count(t[i]))
        {
            v = st[v].next[t[i]];
            ++l;
        }
        if (l > best)
            best = l, bestpos = i;
    }
    return t.substr (bestpos-best+1, best);
}
```

Number of different substrings

We preprocess the string s by computing the suffix array and the LCP array. Using this information we can compute the number of different substrings in the string.

To do this, we will think about which new substrings begin at position p[0], then at p[1], etc. In fact we take the suffixes in sorted order and see what prefixes give new substrings. Thus we will not overlook any by accident.

Because the suffixes are sorted, it is clear that the current suffix p[i] will give new substrings for all its prefixes, except for the prefixes that coincide with the suffix p[i−1]. Thus, all its prefixes except the first lcp[i−1] one. Since the length of the current suffix is n−p[i], n−p[i]−lcp[i−1] new suffixes start at p[i]. Summing over all the suffixes, we get the final answer

Hungarian Algorithm

```
struct Hungarian {
   int cost[N][N];
   int n, maxMatch;
   bool S[N], T[N];
   int lx[N], ly[N], xy[N], yx[N];
   int slack[N], slackX[N], previous[N];

   void initLabels() {
       memset(lx, 0, sizeof(lx));
       memset(ly, 0, sizeof(ly));

       for (int x = 0; x < n; x++) {
           for (int y = 0; y < n; y++)
               lx[x] = max(lx[x], cost[x][y]);
       }
   }

   void updateLabels() {
       int x, y;
       int delta = INF;

       for (y = 0; y < n; y++) {
           if (!T[y])
               delta = min(delta, slack[y]);
       }

       for (x = 0; x < n; x++) {
           if (S[x])
               lx[x] -= delta;
       }

       for (y = 0; y < n; y++) {
           if (T[y])
               ly[y] += delta;
       }

       for (y = 0; y < n; y++) {
           if (!T[y])
               slack[y] -= delta;
       }
   }
```

```
void addToTree(int x, int prev) {
    S[x] = 1;
    previous[x] = prev;

    for (int y = 0; y < n; y++) {
        if (lx[x] + ly[y] - cost[x][y] < slack[y]) {
            slackX[y] = x;
            slack[y] = lx[x] + ly[y] - cost[x][y];
        }
    }
}

void augment() {
    if (maxMatch == n)
        return;

    int x, y, root;
    int q[N], wR = 0, rD = 0;

    memset(S, 0, sizeof S);
    memset(T, 0, sizeof T);
    memset(previous, -1, sizeof previous);

    for (x = 0; x < n; x++) {
        if (xy[x] == -1) {
            S[x] = 1;
            previous[x] = -2;
            q[wR++] = root = x;

            break;
        }
    }

    for (y = 0; y < n; y++) {
        slackX[y] = root;
        slack[y] = lx[root] + ly[y] - cost[root][y];
    }

    while (1) {
        while (rD < wR) {
            x = q[rD++];

            for (y = 0; y < n; y++) {
                if (cost[x][y] == lx[x] + ly[y] && !T[y]) {
                    if (yx[y] == -1)
                        break;
```

```
                    T[y] = 1;
                    q[wR++] = yx[y];

                    addToTree(yx[y], x);
                }

            }

            if (y < n)
                break;
        }

        if (y < n)
            break;

        updateLabels();

        wR = rD = 0;
        for (y = 0; y < n; y++)
            if (!T[y] && slack[y] == 0) {
                if (yx[y] == -1) {
                    x = slackX[y];

                    break;
                }
                else {
                    T[y] = 1;

                    if (!S[yx[y]]) {
                        q[wR++] = yx[y];

                        addToTree(yx[y], slackX[y]);
                    }
                }
            }

        if (y < n)
            break;
    }

    if (y < n) {
        maxMatch++;

        for (int cX = x, cY = y, tY; cX != -2; cX = previous[cX], cY
= tY) {
            tY = xy[cX];
```

```
                yx[cY] = cX;
                xy[cX] = cY;
            }

            augment();
        }
    }

    int hungarian() {
        int ret = 0;

        maxMatch = 0;

        memset(xy, -1, sizeof(xy));
        memset(yx, -1, sizeof(yx));

        initLabels();

        augment();

        for (int x = 0; x < n; x++)
            ret += cost[x][xy[x]];

        return ret;
    }

    int solve(int a[], int s) {
        n = s;

        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                cost[i][j] = -a[i * n + j];

        return -hungarian();
    }
};
```

Max Flow / Min Cut

```
struct MaxFlow {
      int cap[N][N];
      vector<int> g[N];

      void addEdge(int u, int v, int c) {
      cap[u][v] = c;
```

```
g[u].push_back(v);
g[v].push_back(u);
}

int bfs(int s, int t, vector<int>& par) {
fill(par.begin(), par.end(), -1);

queue<pair<int, int>> q;

par[s] = -2;
q.push({s, INF});

while (!q.empty()) {
      int u = q.front().first;
      int flow = q.front().second;

      q.pop();

      for (int v: g[u]) {
            if (par[v] == -1 && cap[u][v]) {
            par[v] = u;

            int newFlow = min(flow, cap[u][v]);

            if (v == t)
                  return newFlow;

            q.push({v, newFlow});
            }
      }
}

return 0;
}

int maxFlow(int s, int t) {
int newFlow, flow = 0;

vector<int> par(n * 2 + 5);

while (newFlow = bfs(s, t, par)) {
      flow += newFlow;

      int cur = t;

      while (cur != s) {
            int prev = par[cur];
```

```
                cap[prev][cur] -= newFlow;
                cap[cur][prev] += newFlow;

                cur = prev;
            }
        }

        return flow;
        }
};
```

Hopcroft-Karp Algorithm

```
struct HopcroftKarp {
    int n, m;
    int neutral = 0;
    vector<vector<int>> g;
    vector<int> to, from, dist;

    HopcroftKarp(vector<vector<int>> &g, int n, int m) : g(g), n(n), m(m)
{
        dist.resize(n + 1);
        to.resize(n + 1, neutral);
        from.resize(m + 1, neutral);
    }

    void addEdge(int u, int v) {
        g[u].push_back(v);
    }

    bool bfs() {
        queue<int> q;

        for (int u = 1; u <= n; u++) {
            if (to[u] == neutral) {
                q.push(u);
                dist[u] = 0;
            }
            else
                dist[u] = INF;
        }

        dist[neutral] = INF;
```

```
    while (!q.empty()) {
        int u = q.front();

        q.pop();

        if (dist[u] < dist[neutral]) {
            for (auto v: g[u]) {
                if (dist[from[v]] == INF) {
                    q.push(from[v]);
                    dist[from[v]] = dist[u] + 1;
                }
            }
        }
    }

    return dist[neutral] != INF;
}

bool dfs(int u) {
    if (u != neutral) {
        for (auto v: g[u]) {
            if (dist[from[v]] == dist[u] + 1) {
                if (dfs(from[v])) {
                    to[u] = v;
                    from[v] = u;

                    return 1;
                }
            }
        }

        dist[u] = INF;

        return 0;
    }

    return 1;
}

int maxMatch() {
    int sol = 0;

    while (bfs()) {
        for (int u = 1; u <= n; u++)
            if (to[u] == neutral && dfs(u))
                sol++;
    }
```

```
        return sol;
    }
};
```

Kuhn Algorithm

```
struct Kuhn {
    int n, m;
    vector<int> mt;
    vector<bool> vis;
    vector<vector<int>> g;

    Kuhn(vector<vector<int>> &g, int &n, int &m) : g(g), n(n), m(m) {
        vis.resize(n);
        mt.assign(m, -1);
    }

    bool match(int u) {
        if (vis[u])
            return 0;

        vis[u] = 1;

        for (auto v: g[u]) {
            if (mt[v] == -1 || match(mt[v])) {
                mt[v] = u;

                return 1;
            }
        }

        return 0;
    }

    int maxMatch() {
        for (int i = 0; i < n; i++) {
            fill(vis.begin(), vis.end(), 0);

            match(i);
        }

        int sol = 0;
        for (int i = 0; i < m; i++)
            sol += mt[i] != -1;
```

```
        return sol;
    }
};
```

Min Cost Max Flow

```
struct Edge {
    int to, cost, cap, flow, backEdge;
};

struct MinCostMaxFlow {
    int n;
    vector<vector<Edge>> g;

    MinCostMaxFlow(int s) {
        n = s + 1;
        g.resize(n);
    }

    void addEdge(int u, int v, int cap, int cost) {
        Edge e1 = {v, cost, cap, 0, (int)g[v].size()};
        Edge e2 = {u, -cost, 0, 0, (int)g[u].size()};

        g[u].push_back(e1);
        g[v].push_back(e2);
    }

    pair<int, int> minCostMaxFlow(int s, int t) {
        int flow = 0, cost = 0;

        deque<int> q;
        vector<int> state(n), from(n), fromEdge(n), d(n);

        while (1) {
            for (int i = 0; i < n; i++)
                state[i] = 2, d[i] = INF, from[i] = -1;

            q.clear();
            q.push_back(s);

            d[s] = 0;

            state[s] = 1;

            while (!q.empty()) {
                int v = q.front();
```

```
            q.pop_front();

            state[v] = 0;

            for (int i = 0; i < (int)g[v].size(); i++) {
                Edge e = g[v][i];

                if (e.flow >= e.cap || d[e.to] <= d[v] + e.cost)
                    continue;

                int to = e.to;

                d[to] = d[v] + e.cost;

                from[to] = v;
                fromEdge[to] = i;

                if (state[to] == 1)
                    continue;

                if (!state[to] || !q.empty() && d[q.front()] > d[to])
                    q.push_front(to);
                else
                    q.push_back(to);

                state[to] = 1;
            }
        }

        if (d[t] == INF)
            break;

        int it = t, addFlow = INF;

        while (it != s) {
            addFlow = min(addFlow,
                          g[from[it]][fromEdge[it]].cap
                          - g[from[it]][fromEdge[it]].flow);

            it = from[it];
        }

        it = t;

        while (it != s) {
            g[from[it]][fromEdge[it]].flow += addFlow;
```

```
                g[it][g[from[it]][fromEdge[it]].backEdge].flow -=
addFlow;

                cost += g[from[it]][fromEdge[it]].cost * addFlow;

                it = from[it];
            }

            flow += addFlow;
        }

        return {cost, flow};
    }
};
```

Probability

Ncr Iterative (Without Mod)

```
for (int i=0;i<=100;i++)
   {
       ncr[i][0]=ncr[i][i]=1;
       for (int k=1;k<i;k++)
       {
           ncr[i][k]=ncr[i-1][k]+ncr[i-1][k-1];
       }
   }
// max 100
```

Notes and rules

In some cases, the first event happening impacts the probability of the second event. We call these **dependent events**.

In other cases, the first event happening does not impact the probability of the seconds. We call these **independent events**.

Conditional Probability

$$P(\text{A and B})=P(\text{A})\cdot P(\text{B}\mid\text{A})$$

If the events are independent, one happening doesn't impact the probability of the other, and in that case $P(\text{B}\mid\text{A})=P(\text{B})$

Expected Value

E(x)=p(x)*x
E(x+y)=E(x)+E(y)
E(x*y)=E(x)*E(y)
E(x+c)=E(x)+c
E(x*c)=E(x)*c
For continuous random variable E(x)=$\int_x$ x*f(x) dx (from zero to x)
Conditional expected value
$E[X\mid Y=y]=\sum_x P(X=x\mid Y=y)$.

Linear Expextation

$E[1_t]=P(t \text{ occurs})\cdot 1+P(t \text{ does not occur})\cdot 0=P(t \text{ occurs})$.

Infinite Probability

p(x)/=(1-p(x));